

Advanced Time Series - Computer Exercises 3

Authors

Gunn P. Jacobsen - s175799
Linda Elisabet á Geilini - s223116

Group: 09

September 18, 2026

Contents

1 Part 1 - Simulation and discretization of diffusion processes **1**
1.1 Question 1.1 1
1.2 Question 1.2 3

2 Part 2 - Modeling of Rainfall-runoff **5**
2.1 Part 2.1 - Linear rainfall-runoff 5
2.2 Part 2.2 - Non-linear rainfall-runoff with overflowr 9
2.3 Part 2.3 - Larger real-life case 14
2.1 Question 2.3.2 16

3 Appendex A **21**
3.1 Plot of observations from the larger real life rainfall-runoff case 21
3.2 Code 26

1 Part 1 - Simulation and discretization of diffusion processes

In this first section we are considering a two-dimensional system which aims to describe the firing of a single neuron. [1] The equations are

$$\frac{dx_t^1}{dt} = \theta_3 \left(x_t^1 + x_t^2 - \frac{1}{3}(x_t^1)^3 + \theta_4 \right) \tag{1a}$$

$$\frac{dx_t^2}{dt} = -\frac{1}{\theta_3} \left(x_t^1 + \theta_2 x_t^2 - \theta_1 \right) \tag{1b}$$

where x_t^1 is the negative potential over the membrane and x_t^2 is the permeability of the membrane. Imperfections and surrounding neuron activity can be simulated by adding small noise to 1a, resulting in a two-dimensional system of Itô stochastic differential equation which further can be discretize using Euler-Marayama approximation. We then arrive at the equations

$$\begin{aligned} Y_{n+1}^1 &= Y_n^1 + \theta_3 \left(Y_n^1 + Y_n^2 - \frac{1}{3}(Y_n^1)^3 + \theta_4 \right) \Delta + \sigma \Delta W_{n+1}^1 \\ Y_{n+1}^2 &= Y_n^2 - \frac{1}{\theta_3} \left(Y_n^1 \theta_2 Y_n^2 - \theta_1 \right) \Delta \end{aligned} \tag{2}$$

where W_t is a standard Wiener process and $\sigma > 0$ is the incremental standard deviation. Δ is a suitably chosen small time interval and $W_{n+1} \in N(0, \Delta)$. Lastly we have that (Y_n^1, Y_n^2) is a discrete approximation of (X_t^1, X_t^2) in the time interval $0 \leq t \leq T$

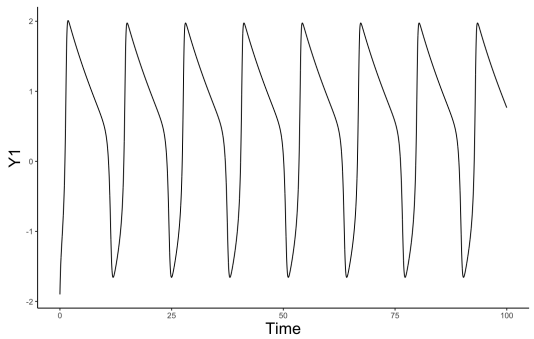
We have been given that when the physical parameters $(\theta_1, \theta_2, \theta_3, \theta_4) = (0.7, 0.8, 3.0, -0.34)$ the two-dimensional system given in equations 1 has a limit cycle.

1.1 Question 1.1

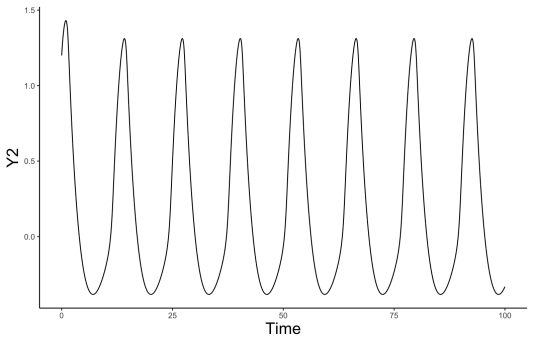
We now simulate the discretized system given in 2 over the time interval $0 \leq t \leq T$. The parameters are set as $(\theta_1, \theta_2, \theta_3, \theta_4) = (0.7, 0.8, 3.0, -0.34)$, $\Delta = 2^{-9}$, and $(Y_0^1, Y_0^2) = (-1.9, 1.2)$. The system is simulated for five different values of σ ranging from 0 to 0.4. For consistency, the same noise from the Wiener process is used across all simulations, to ensure that observed differences in behavior arise only from variations in σ. A plot of the simulated realizations of Y^1 and Y^2 for the different values of σ is presented in Figure 1, while Figure 2 shows the corresponding phase plots for these simulations.

Before discussing the observations from these figures, recall that a limit cycle is an isolated closed trajectory (or closed orbit) in the phase space of a dynamical system. This characteristic is reflected in Figure 2(a), where no noise ($\sigma = 0$) is added to the system. The perfect periodic behavior associated with this limit cycle is also seen in Figures 1(a) and 1(b). As σ increases, we observe that the trajectories begin to fluctuate more and the system shows a more and more irregular-periodic behavior, where the oscillations are still present but with some variability in amplitude and phase. Therefor the concept of a limit cycle becomes less meaningful due to the loss of an isolated closed orbit in the phase space.

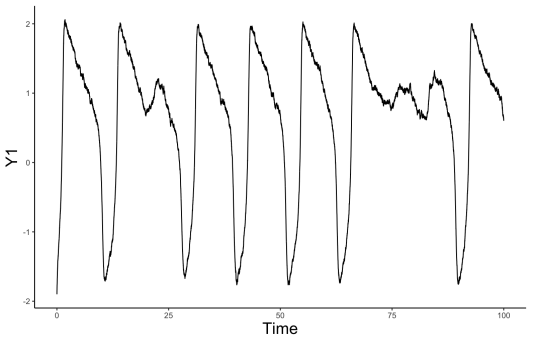
[1] The system is a simplification of a four- dimensional system of ordinary differential equations



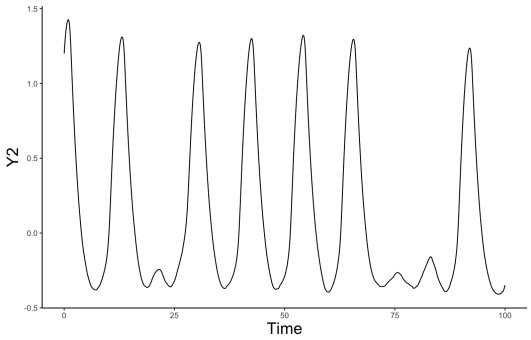
(a) Y_1 with $\sigma = 0$



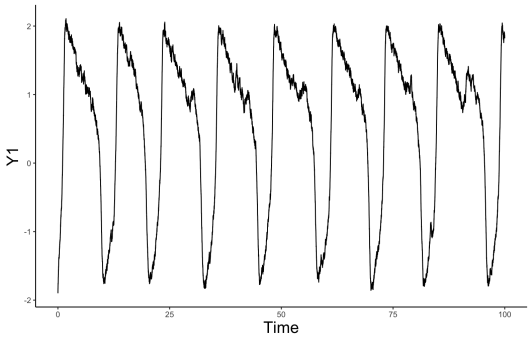
(b) Y_2 with $\sigma = 0$



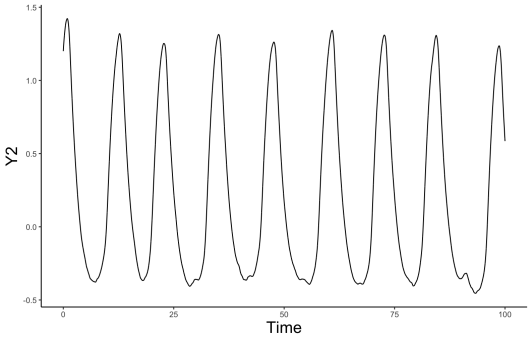
(c) Y_1 with $\sigma = 0.1$



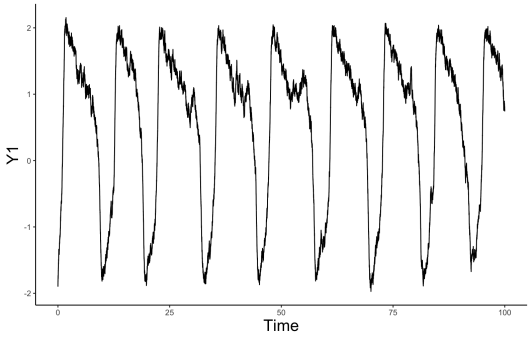
(d) Y_2 with $\sigma = 0.1$



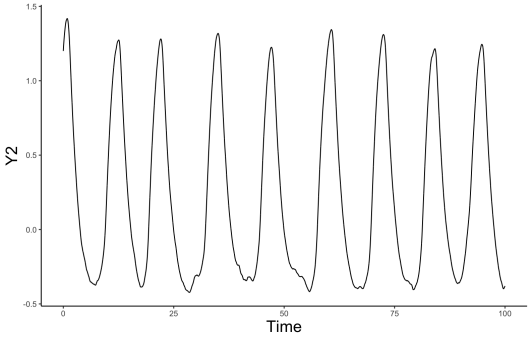
(e) Y_1 with $\sigma = 0.2$



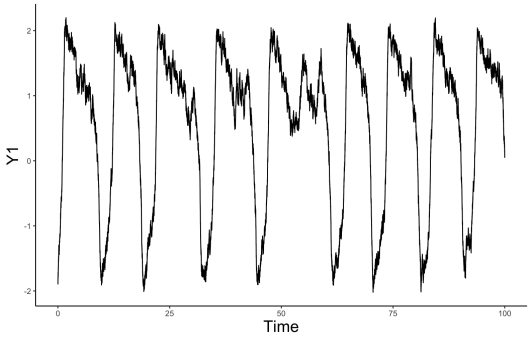
(f) Y_2 with $\sigma = 0.2$



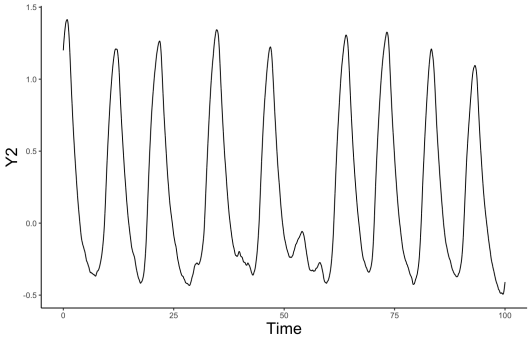
(g) Y_1 with $\sigma = 0.3$



(h) Y_2 with $\sigma = 0.3$

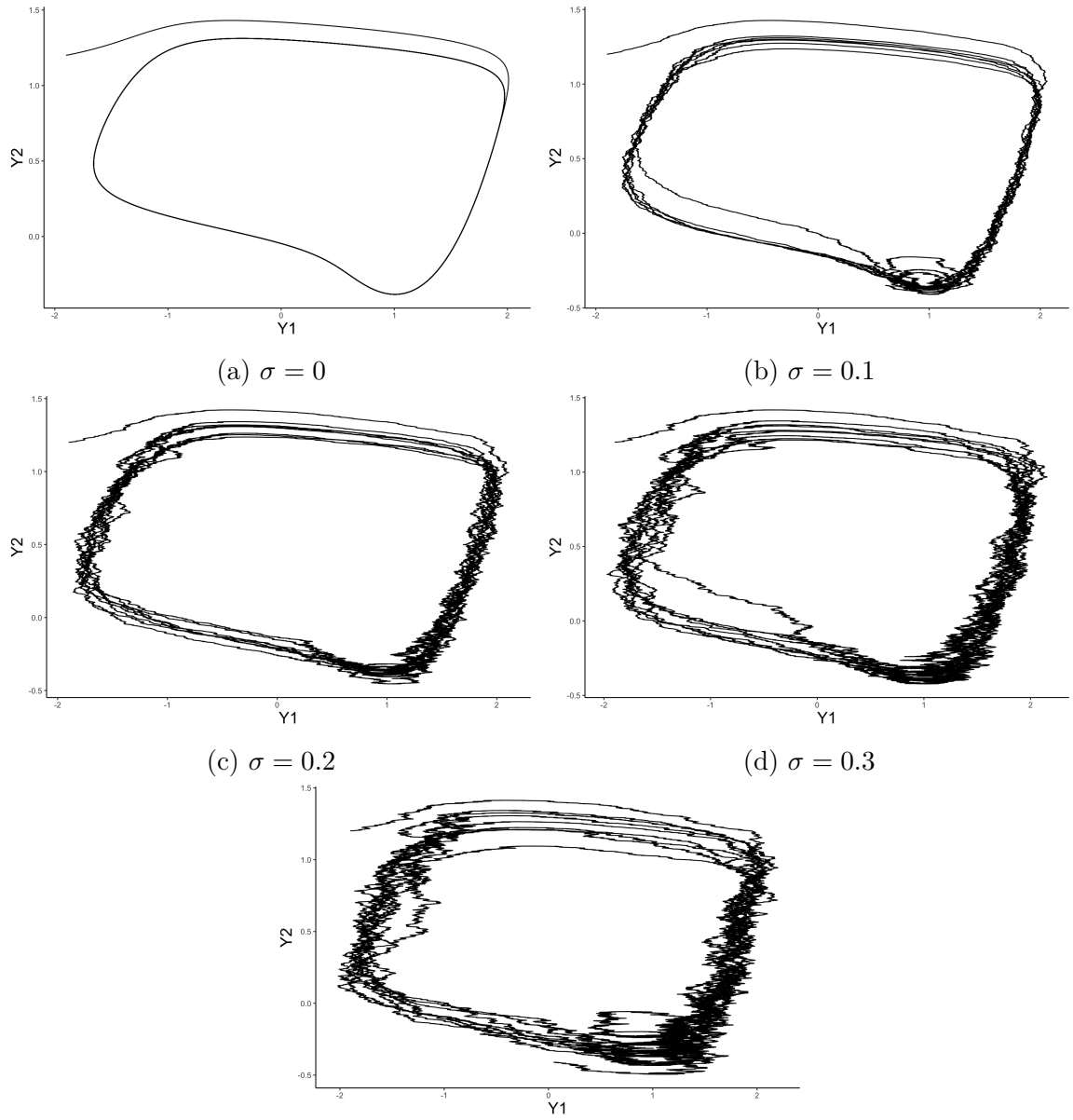


(i) Y_1 with $\sigma = 0.4$



(j) Y_2 with $\sigma = 0.4$

Figure 1: Realizations of Y^1 and Y^2 with 5 different values for σ



(a) $\sigma = 0$

(b) $\sigma = 0.1$

(c) $\sigma = 0.2$

(d) $\sigma = 0.3$

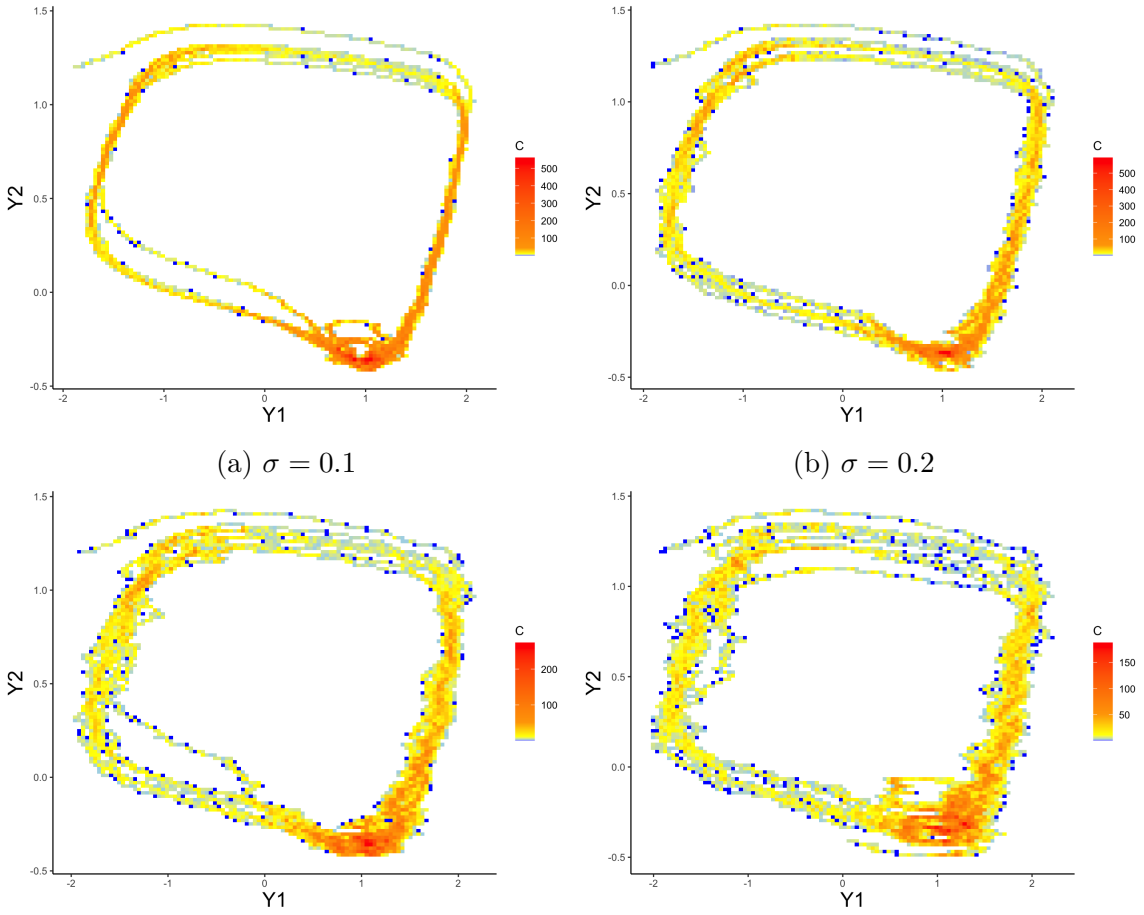
(e) $\sigma = 0.4$

Figure 2: Phase-plot of (Y^1, Y^2) from the realizations plotted in Figure 1

1.2 Question 1.2

In this question, we partition the phase plane into 100×100 equal cells and count the number of trajectories passing through each cell to construct a 2D-histogram plot. We do this because, in a stochastic system compared to a deterministic one, the attractor becomes a probability distribution rather than a limit cycle. A histogram plot can provide information about the form and support of the probability density function for the asymptotically stable, stationary solution of the system.

The 2D-histogram plots for the simulated realizations from the previous question with $\sigma \in \{0.1, 0.2, 0.3, 0.4\}$ are shown in Figure 3. From the plot, we observe that the probability density has ring-shaped support, with the thickness of the ring depending on σ. This ring-shaped structure was also observed in the standard phase plots in Figure 2. However, the 2D-histogram provides additional information about the shape of the density function, specifically where within the density region the system is most likely to be



(a) $\sigma = 0.1$ (b) $\sigma = 0.2$

(c) $\sigma = 0.3$ (d) $\sigma = 0.4$

Figure 3: 2D histogram plot of the simulated realizations with different values of σ, where the phase plane is partitioned into 100×100 equal cells.

found. For example, Figure 3 shows that the system is most frequently found in the lower right corner, indicating that the probability density function is highest in this area. This occurs when the negative potential across the membrane is high and the membrane permeability is low. We also observe that the density function takes relatively high values in the upper left corner, though not as high as in the lower right corner.

2 Part 2 - Modeling of Rainfall-runoff

In this second part, we will attempt to model rainfall-runoff using stochastic differential equations. We are given that, in the linear case, the runoff process can be approximated as the discharge of water between a finite number of reservoirs. The system of stochastic differential equations is given by:

$$\begin{aligned} dX_{1,t} &= AU_t dt - \frac{n}{K} X_{1,t} dt + \sigma dw_{1,t} \\ dX_{2,t} &= \frac{n}{K} X_{1,t} dt - \frac{n}{K} X_{2,t} dt + \sigma dw_{2,t} \\ &\vdots \\ dX_{n+1,t} &= \frac{n}{K} X_{n,t} dt + \sigma dw_{n+1,t} \end{aligned} \tag{3}$$

where U_t is the rainfall intensity, A is the area where rainfall is collected, and K is related to the time scale of the runoff process. Additionally, n represents the number of states (or the number of reservoirs) minus 1, and σ is a noise parameter

2.1 Part 2.1 - Linear rainfall-runoff

Question 2.1.1

To formulate a linear reservoir model with 2 states, we use the system equations in 3 with $n = 1$, resulting in the following model:

$$\begin{aligned} dX_{1,t} &= AU_t dt - \frac{1}{K} X_{1,t} dt + \sigma dw_{1,t} \\ dX_{2,t} &= \frac{1}{K} X_{1,t} dt + \sigma dw_{2,t} \end{aligned} \tag{4}$$

We are given that the observations correspond to the second state, i.e., $Y_{t_i} = X_{2,t_i} + e_{t_i}$, where e_{t_i} is the observation noise.[2]. The parameters to be estimated are $\boldsymbol{\theta} = \{K, A, \sigma\}$, where U_t is an input.

We estimate the parameters using the ctsmTMB package in R, and we are applying the Extended Kalman Filter method. We model the system with the time unit in hours. The estimates, along with their variances and the initial values as well as lower and upper bounds specified for the parameter estimation, are presented in Table 1. It is reasonable to restrict both K and A to be positive, as negative values for these parameters do not have any meaningful physical interpretation. We have also set the initial value of the two states to be 0 since we assume that both basins are empty to start. The initial uncertainty for this assumption is set to 0.1. The time constant related to the runoff process is estimated to be 3.0, while the catchment area is estimated to be 6.3. The negative log-likelihood for the model with the estimated parameters is 218.2.

Parameters	**Estimate**	**Variance**	**Initial**	**Lower**	**Upper**
K	3.0	0.08	1	1e-10	50
A	6.3	0.22	1	1e-10	50

Table 1: Estimates of K and A from the linear model with 2 states using ctsmTMB

[2]Observations are available at the time points t_i : $0 = t_0 < \cdots < t_i < \cdots < t_N = 10$ where $N = 601$

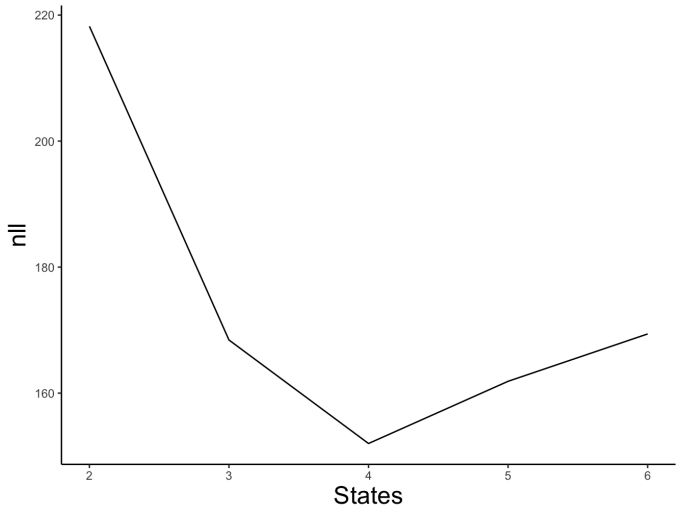


Figure 4: Log-likelihood of models as the number of states increases

Question 2.1.2

We now aim to determine the most suitable number of states for the model. To achieve this, we extend model 4 by adding one state at a time. In particular, we consider the system given in 3 for $n \in \{1, 2, 3, 4, \dots\}$. For each of the extended models, we use the same method, the same initial values, as well as the same lower and upper bounds for parameters A and B, as were used for the two-state model. Additionally, we maintain the initial assumption that all states start at zero with an uncertainty of 0.1. By evaluating the negative log-likelihood for each model, we select the model with the lowest negative log-likelihood as having the most suitable number of states. Figure 4 illustrates how the log-likelihood changes as the number of states increases. Based on the plot, we observe that the model with 4 states provides the best fit, with a negative log-likelihood of 152.0, compared to 218.2 for the two-state model.

Question 2.1.3

The estimated parameters for the 4-state model, which was found to be the most suitable, are shown in Table 2.

Parameters	**Estimate**	**Variance**	**Initial**	**Lower**	**Upper**
K	2.1	0.001	1	1e-10	50
A	5.1	0.010	1	1e-10	50

Table 2: Estimates of K and A from the linear model with 4 states using ctsmTMB

Comparing the variances of the parameters of the two models, we can see, that the variances are much smaller for the 4-state model - indicating higher confidence in the estimates. Higher confidence in the estimates suggest a more reliable model, which is also what we would expect because the 4-state model had a lower nll value.

Comparing with the 2-state model we can see, that we have a decrease in the estimate of K from 3.0 to 2.1. The parameter K is related to the time scale of the runoff, a lower K results in a faster runoff process. The decrease of K is because in both models we have the final state as the observation state. Our model is bounded by time as we need to have (or at least try to have) the correct water-level in the final state at each time point. This means, that in a 4-state model, we need the water to travel faster

through each state compared to a 2-state model.

If we had fixed K to be the same in both models, we would expect A to increase when moving from the 2-state to the 4-state model. This is because A scales the rainfall input U_t, and adding reservoirs typically reduces system efficiency (meaning water moves more slowly through the system). So for the model to maintain the correct water level in the observation state, more water would need to be added to the system. However, we observe that A decreases from 6.3 to 5.1. This is probably related to the observed decrease in K so the system has not become less efficient despite the additional states.

Question 2.1.4

The correlation matrix for K and A in the 4-state model is the following:

$$\begin{array}{c} \\ K \\ A \end{array} \begin{array}{c} \begin{array}{cc} K & A \end{array} \\ \begin{pmatrix} 1 & 0.41 \\ 0.41 & 1 \end{pmatrix} \end{array}$$

We can see from the correlation matrix, that the parameters K and A have a moderate correlation of 0.41. This tells us, that when K increases (or decreases), A tends to do the same. This corresponds to what we saw in the previous question, where both A *and* K decreased, when going from the 2-state to 4-state model. From a physical aspect this tells us, that if K decreases and the water starts flowing faster through the system, then the model compensates by also reducing A as the model doesn't need as much input at each time step to achieve the same water level in the observation state. While if K increases and water flows slower, then A will also increase, as the model would then need a larger input.

Question 2.1.5

We perform residual analysis on the 4-stae model using the ctsmTMB package and get the plots in Figure 5. Looking at the Time Series of the residuals, we can see that apart from the first few hours, where the residuals are zero, the residuals seem fairly randomly distributed. There does not appear to be a systemic error or trend in the residuals.
The histogram shows a large spike at zero, which indicates that the residuals are centered around zero, the rest of the residuals seem to follow a normal distribution around 0. This suggests that the model might deviate slightly from a regular normal distribution. The QQplot mostly follows the diagnoal line, with some deviations at the tails. This suggests that the residuals are approximately normally distributed, but that we have some that there is some non-normality in the extremes. The cumulative periodogram we can see that the residuals are mostly within the bounds, which further supports the fact that the residuals are random. Finally we see that the ACF and PACF mostly look like white noise.

Taking all the residual plots into considerations, the residuals appear to be approximately normally distributed. The model and observed data are plotted together in Figure 6, and we see that the model closely follows the trend of the observations with minimal deviation.

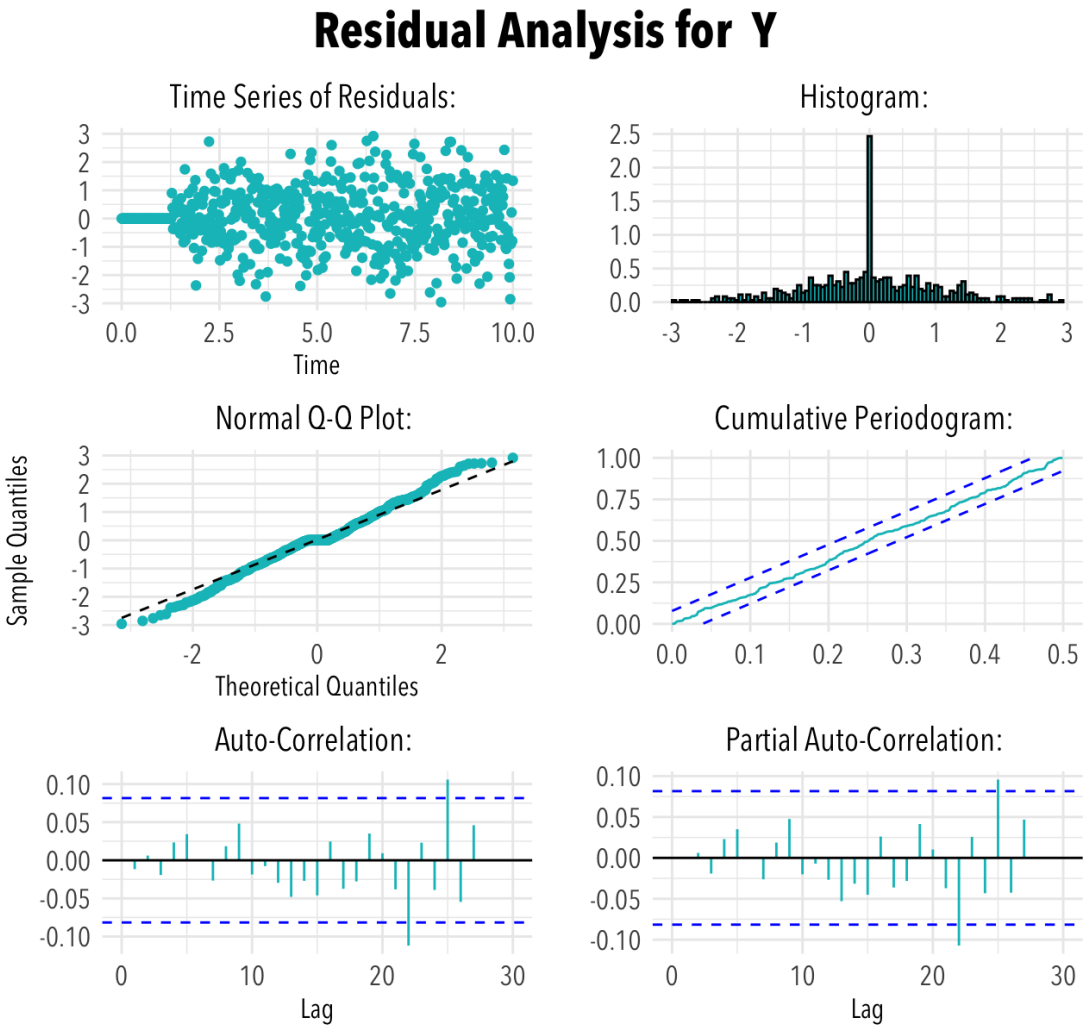


Figure 5: Residual plots for a linear model with 4 states

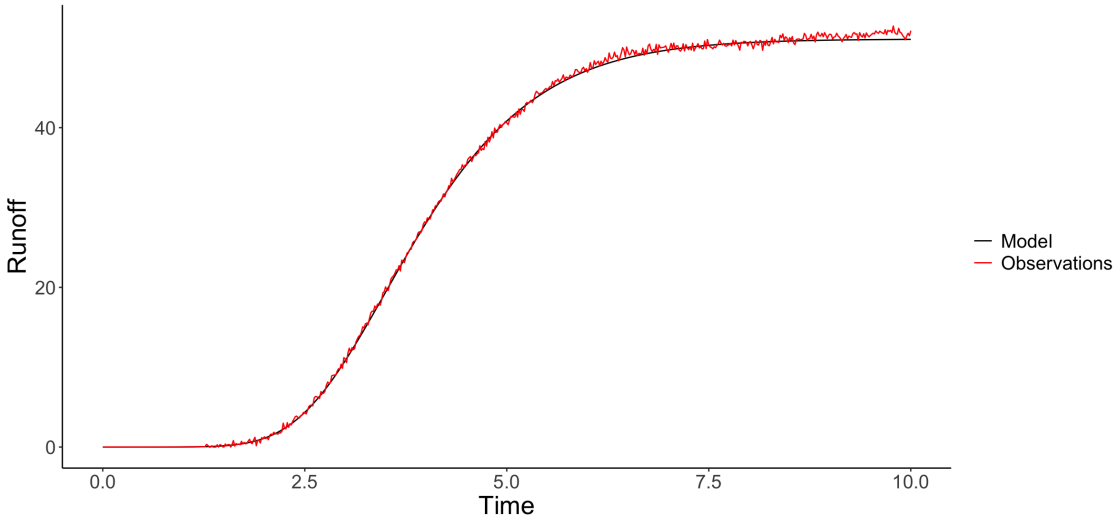


Figure 6: Linear model with 4 states plotted together with the observations

2.2 Part 2.2 - Non-linear rainfall-runoff with overflowr

Question 2.2.1

We now introduce overflow to our system and our observation is the overflow state. This means that when a certain amount of stormwater has accumulate in the system it starts flowing into the observed state from another earlier state. The stormwater can therefore discharge in two ways. It is always dischargin further downstream at the usual rate, and when the overflow threshold is met it is also discharging into the side facility, where the measurements are taken.

We formulate a linear reservoir of the model with three states, describing the overflow problem. Where the third state is the observed state. The model is given as follows:

$$dX_{1,t} = A \cdot U_t dt - \frac{2}{K} \cdot X_{1,t} dt \cdot \sigma \cdot dw_{1,t} \tag{5}$$

$$dX_{2,t} = \frac{2}{K} \cdot X_{1,t} dt - \frac{2}{K} \cdot X_{2,t} dt - \frac{2}{K_{overflow}} \cdot (X_{2,t} - \beta) dt \cdot \sigma \cdot dw_{2,t} \tag{6}$$

$$dX_{3,t} = \frac{2}{K_{overflow}} \cdot (X_{2,t} - \beta) dt \cdot \sigma \cdot dw_{3,t} \tag{7}$$

The formulation is similar to the one we saw in part 2.1, but the main difference is that we have included the overflow discharge in the second state. The overflow discharge is modeled similarly to the downstream discharge, but we have a different parameter $K_{overflow}$, which determines the timescale of the overflow discharge process, which isn't necessarily the same as the downstream discharge process. More importantly we have included a parameter β in the differential equation for state 2 and 3. β is our threshold parameter and it is only when $X_{2,t} > \beta$ that the overflow discharge happens.

We estimate the model using the ctsmTMB package. We require β to be positive, because it is related to the threshold, and if β, was negative it would mean the threshold was exceeded, when we had a negative effect - this wouldn't make physical sense. The rest of the initial guesses, lower and upper bounds for the parameters are the same as in part 2.1. We get a negative logliklihood value of -1413.24 and the estimated parameters are shown in Table 3

Parameters	**Estimate**	**Variance**	**Initial**	**Lower**	**Upper**
K	3.7	0.139	1	1e-10	50
$K_{overflow}$	0.54	0.033	1	1e-10	50
A	1.59	0.014	1	1e-5	50
β	0.78	0.05	4	0	10

Table 3: Estimates of K, $K_{overflow}$,A and β from the linear model of the overflow problem

On Figure 7 the data together with the predicted values of the model are plotted, to give us a visual idea of how good the fit of the model is. The figure clearly shows why it isn't optimal to model the overflow problem with a linear model. The data doesn't behave linearly — constant at zero until about 2.6 hours, then suddenly increasing. A linear model cannot capture this sudden change and instead smooths it out.
We notice the model initially underestimates and later overestimates the observations due to its structure. When $x_{2,t} < \beta$ the difference difference $(x_{2,t} - \beta)$ is negative this makes the term $\frac{2}{K_{overflow}} \cdot (X_{2,t} - \beta)$ negative in the equation for state 3, leading to initial negative observations.

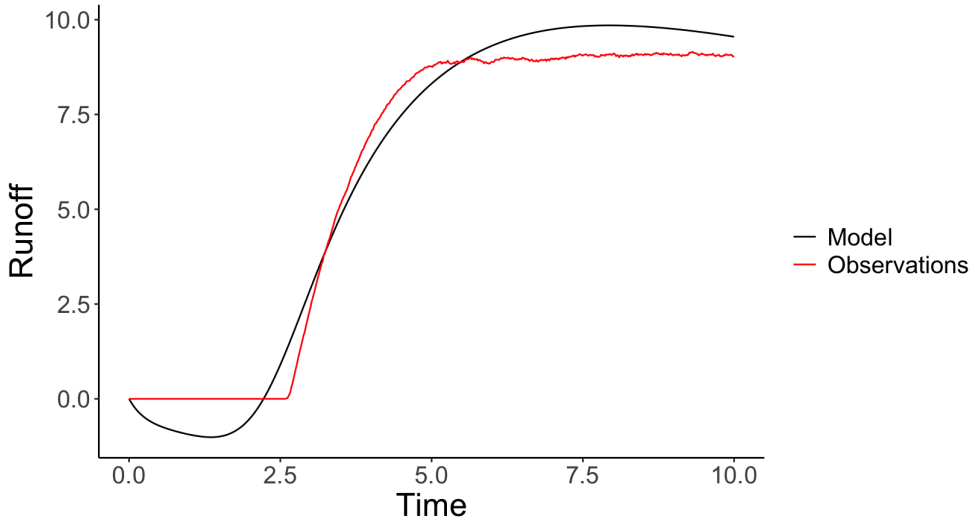


Figure 7: Data plotted together with predictions made using the model

We observe that the model predicts the overflow to start before the overflow actually begins. This is because before the threshold β is reached, $(X_{2,t} - \beta) < 0$, resulting in the term $\frac{2}{K_{overflow}} \cdot (X_{2,t} - \beta)$ being positive in the equation for state 2 - Adding extra water to state 2 and causing it to reach the threshold sooner than it should. This added inflow lacks physical sense and highlights the problem of modeling an on-off switch with a linear model.

After 5 hours, when the observed data stabilizes, the model overestimates the observations. This may be because the model struggles to drop below the threshold, even when the rain intensity stops, due to the added inflow in state 2 when $(x_{2,t} - \beta) < 0$. It could also be that all the small discrepancies between the actual system and our model accumulate over time. After 5 hours these accumulated errors, could lead to the observed overestimation.

Question 2.2.2

In order to model the overflow problem more properly we introduce the sigmoid function: $\frac{1}{1+e^{-\alpha(1-\beta)}}$ that takes on values within the range $(0, 1)$, meaning it can act as the on-off switch for the overflow, that we were missing in the previous model. The parameter β acts as the threshold, and is the value that determines, when the function begins transitioning from 0 to 1, while α determines how sharp the transition is.

When including the sigmoid function, the model of the overflow problem is as follows:

$$dX_{1,t} = A \cdot U_t dt - \frac{2}{K} \cdot X_{1,t} dt \cdot \sigma \cdot dw_{1,t} \tag{8}$$

$$dX_{2,t} = \frac{2}{K} \cdot X_{1,t} dt - \frac{2}{K} \cdot X_{2,t} dt - \frac{2}{K_{overflow}} \cdot X_{2,t} \cdot \frac{1}{1 + e^{-\alpha(X_{2,t} - \beta)}} dt \cdot \sigma \cdot dw_{2,t} \tag{9}$$

$$dX_{3,t} = \frac{2}{K_{overflow}} \cdot X_{2,t} \cdot \frac{1}{1 + e^{-\alpha(X_{2,t} - \beta)}} dt \cdot \sigma \cdot dw_{3,t} \tag{10}$$

Like before, we estimate the model using the ctsmTMB package. We get a negative loglikelihood value of -1632.48, which is smaller that the negative loglikelihood value of the linear model(-1413.24) - indicating a better model. The estimated parameters are shown in Table 4
Since β represents the threshold for overflow, we set its lower bound to 0 because a negative threshold

Parameters	**Estimate**	**Variance**	**Initial**	**Lower**	**Upper**
K	4.04	0.01	1	1e-10	50
$K_{overflow}$	1.14	0.71	1	1e-10	50
A	3.26	0.005	1	1e-5	50
α	10.48	22.21	1	0	50
β	6.96	0.11	0.5	0	50

Table 4: Estimates of K, $K_{overflow}$,A, α and β from the non-linear model of the overflow problem

lacks physical meaning in this context. If $\alpha < 0$ the sigmoid function flips, meaning it decreases instead of increasing as X_2 increases. This does not accurately reflect our system, as we expect the overflow from the second state to be larger when X_2 is significantly beyond the threshold compared to when it is only slightly beyond the threshold. Therefor we also set the lower bound for α to be 0.

Question 2.2.3

The correlation matrix of the non-linear model is:

$$\begin{array}{c} \\ K \\ K_{overflow} \\ A \\ \alpha \\ \beta \end{array} \begin{array}{c} \begin{array}{ccccc} K & K_{overflow} & A & \alpha & \beta \end{array} \\ \begin{pmatrix} 1 & 0.41 & -0.22 & 0.60 & -0.41 \\ 0.41 & 1 & -0.11 & 0.86 & -0.56 \\ -0.22 & -0.11 & 1 & -0.24 & 0.78 \\ 0.60 & 0.86 & -0.24 & 1 & -0.65 \\ -0.41 & -0.56 & 0.78 & -0.65 & 1 \end{pmatrix} \end{array}$$

When considering the correlation matrix, we are mostly interested in the moderate to high correlations as these are the ones, that can affect the reliability of our parameter estimation and interpretation.

When considering the parameter K, we see it has a moderate positive correlation of 0.41 and 0.60 with $K_{overflow}$ and α, respectively. K also has a moderate negative correlation of -0.41 with β. This means that the processes that control the downstream discharge and overflow discharge are somewhat linked. When K is large the water lakes longer to discharge downstream, this tends to result in slower overflow discharge(large $K_{overflow}$) as well. A high K also increases the sharpness of the sigmoid function (α), but decreases the threshold, β. This makes sense as slower downstream discharge leads to water accumulating in the system, but the model compensates by making a sharper and sooner overflow activation. This is to ensure that the overflow starts at the correct time.

The parameter $K_{overflow}$ is has a strong positive correlation of 0.86 with α and a moderate negative correlation of -0.56 with β. This means that when the overflow discharges at a slower rate, the model compensates by lowering the threshold and making the overflow transition sharper. Like before, this is because the model is trying to match the timing and duration of the overflow to the data.

Finally we see that the parameter β has a strong positive correlation of 0.78 with A and a strong negative correlation of -0.65 with α. This means, that when the catchment area, A, is larger, the model increases the threshold, likely due to the system reaching higher water levels faster, and the model is trying to delay the overflow to match the data. The correlation with α means if there is a sharp overflow transition, the model compensates by lowering the threshold to ensure accurate timing and progression of the overflow.

It is concerning that we have so many moderate to high correlations between parameters, as it makes it harder to determine which effects are caused by specific parameter. This can make it difficult to uniquely estimate parameters from the data. Correlated parameters can lead to higher variances for the parameters, making the estimation less reliable. This could i.e. be the reason α's large variance. Unreliable parameters can lead to unreliable model predictions.

Question 2.2.4

We perform residual analysis using the ctsmTMB package and get the plots in Figure 8

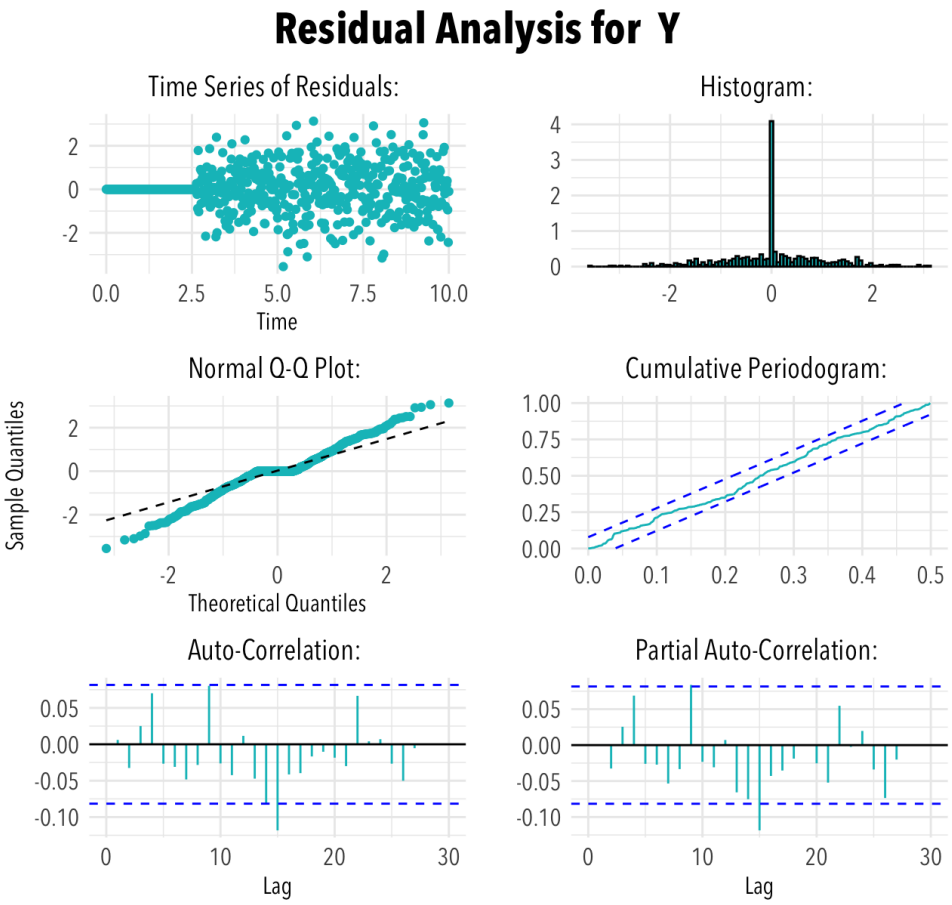


Figure 8: Residual Analysis of the non-linear overflow model

Looking at the Time Series of the residuals, we can see that apart from the first few hours (where the residuals are zero) they seem to be fairly randomly distributed, with no indication of systematic structure or trends. This suggests that the model is good at capturing the systematic structure of the data.
The histogram has a large spike around zero and a narrow distribution. This could suggest that the residuals are concentrated around the mean, but slightly deviate from a regular normal distribution. The Q-Q plot indicates that the residuals are fairly normally distributed, but has some deciation in the tails. They seem to be approximately normally distributed, but there are issues in the extremes. The cumulative preiodogram, the ACF and the PACF all look good, where the values mostly seem to stay within the bounds. Based on all plots combined, we can conclude, that the residuals are approximately normally distributed.

We now plot the observed data together with the predicted values from the model and get the plot in Figure 9.

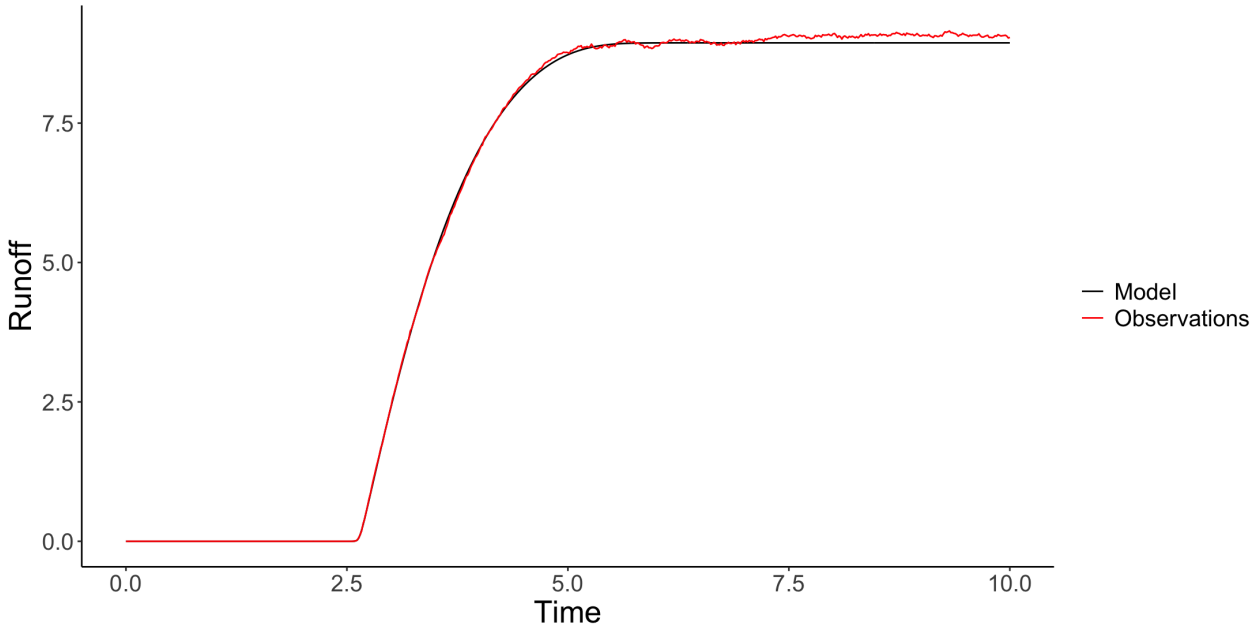


Figure 9: Observed data together with the predicted values from the model.

Compared to the linear model, the current model has significantly improved the ability to predict the data, with only minor deviations observed once the overflow process stabilizes, which is approximately five hours into the simulation. While we are largely satisfied with the model's performance, there is still room for further improvement, so that the model might be able to better capture the system's dynamics after the model has stabilized.

2.3 Part 2.3 - Larger real-life case

For this part we will be dealing with a system, where rainfall travels over the Ground Surface and then flows into a Combined Sewer System. In the Combined Sewer System there are two methods of discharge: Further downstream runoff or the water can overflow into a Stormwater Tunnel. At the end of the tunnel there is a Storage Tower. Water from the Storage Tower will be pumped out of the Storage Tower and further downstream. The Storage Tower has a maximum capacity of $29000m^3$.

Question 2.3.1

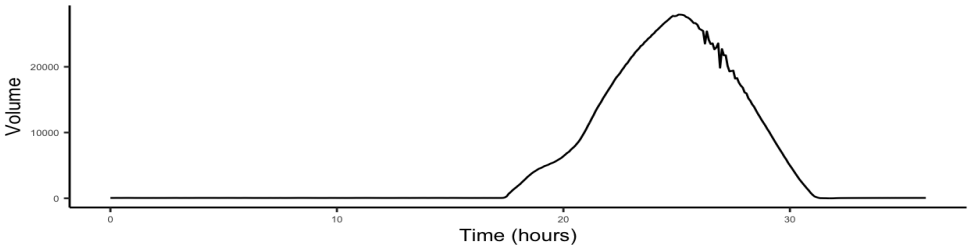
We consider the dataset `ex3 largecase.csv`. It contains 6 different stormwater events from the year 2018, each event spans over 15-36 hours. For each event we have three types of measurements. There is the volume of stormwater inside a Storage Tower. These measurements will be treated as our response variable during the modeling process. We also have measurements of the rainfall and the pumpflow, which will be treated as inputs. All data is provided in 5-minute resolution. The Volume measurements are all instantaneous, while the Rainfall and Pumpflow measurements are averaged values over 5 minutes. The dataset also contains the attributes Timestamp, which is given as the UTC time and Event ID, that indicates which event the observations are for.

We plot the data with the x-axis in hours. The volume plots are shown in Figure 10. Additionally, the volume plot, along with the corresponding rainfall and pump flow for each event, is shown in Figures 13 to 18 in the appendix. Examining the events, rainfall varies between them, causing differences in the Storage Tower's volume. In Figure 10a, event 1's stormwater volume approaches the tower's maximum capacity, while event 2 (Figure 10b) is unique in showing two volume peaks. Only in event 2 (14a) do we observe a rainfall break.

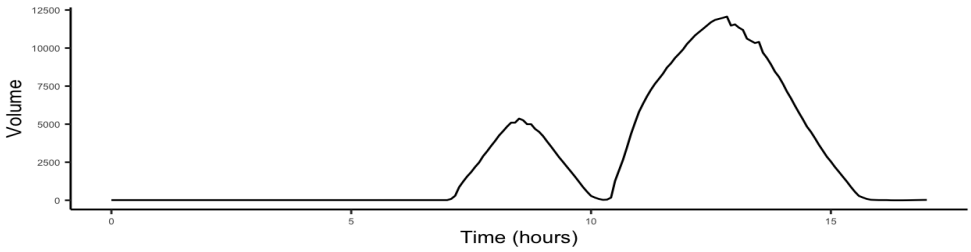
Comparing rainfall and volume plots across events, we note a delay between rainfall and volume increase. This occurs because the Storage Tower only receives overflow after the Combined Sewer System's threshold is exceeded. Comparing Pumpflow and Volume plots, the pump does not activate immediately when water enters the tower. There is no clear volume threshold for pump activation, and the cause of activation is unclear from the plots.

At the start of volume decrease, we see more fluctuation compared to later stages or the volume increase. This fluctuation may result from simultaneous inflow and outflow. The pump actively removes water, but residual inflow may still occur. Finally, the pumpflow stops once the Storage Tower empties, which is expected.

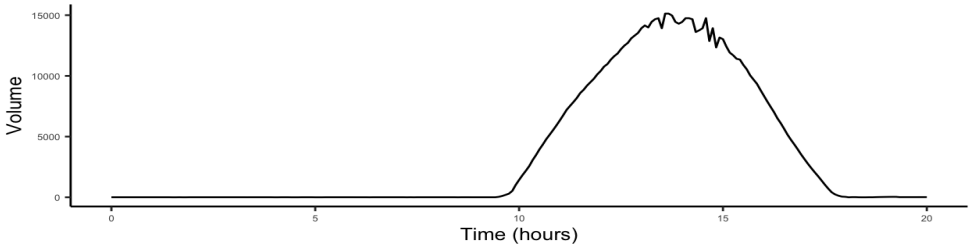
Although much of the system's structure is similar to that in Parts 2.1 and 2.2, there may be challenges in modeling this case. One of the main challenges is modeling the path of the water after it overflows from the Combined Sewer System into the Stormwater Tunnel and until it is pumped out of the Storage Tower further downstream. This is particularly difficult because we need to determine what happens to the water once it is inside the Stormwater Tunnel. Specifically, we must decide whether the Stormwater Tunnel and Storage Tower should be modeled as separate states, how to ensure the water level does not exceed the Storage Tower's maximum capacity. Additionally, it is necessary to account for water that cannot be accommodated in the Storage Tower, such as whether it overflows or remains in the Stormwater Tunnel. Another challenge will be how to incorporate the pumpflow into the system effectively.



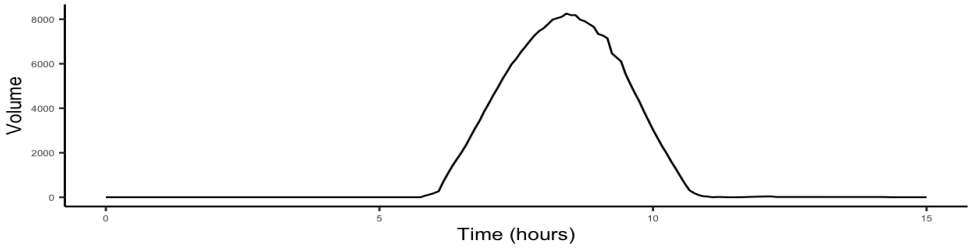
(a) Event 1 with t_0 = 2018-08-12 04:00:00



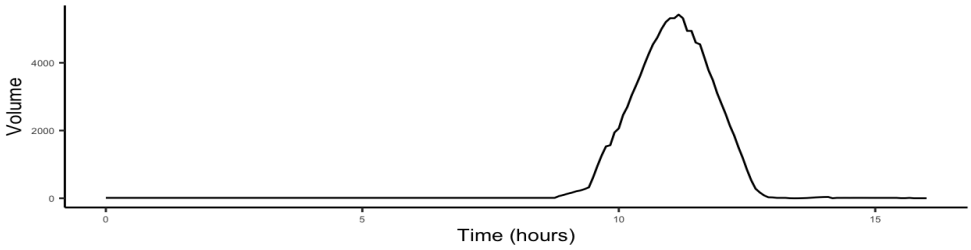
(b) Event 2 with t_0 = 2018-08-27 07:00:00



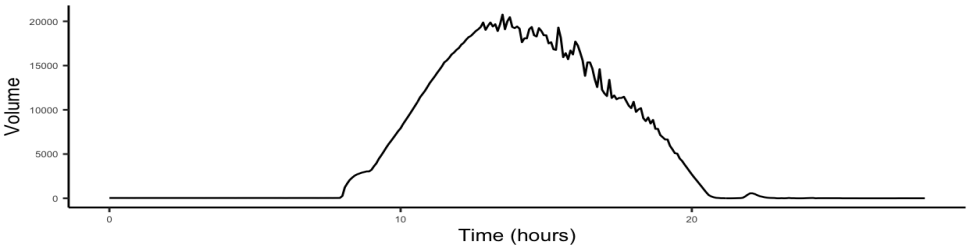
(c) Event 3 with t_0 = 2018-08-30 04:00:00



(d) Event 4 with t_0 = 2018-10-06 21:00:00



(e) Event 5 with t_0 = 2018-11-11 00:00:00



(f) Event 6 with t_0 = 2018-12-21 12:00:00

Figure 10: Observations for stormwater volume for all 6 events

2.1 Question 2.3.2

We now try to model the described system. We define the dynamics of state $X_{1,t}$, which corresponds to the Ground Surface, the same way we did in the other models. $X_{2,t}$ will correspond to the Combined Sewer System. The Combined Sewer System has two ways of water discharge - regular downstream runoff and potential overflow. We will therefore define the dynamics of $X_{2,t}$ the same way as we defined $X_{2,t}$ in the non-linear case in Part 2.2, where the sigmoid function acted as a on-off switch for the overflow.

The third state $X_{3,t}$ will be the Stormwater Tunnel. The Stormwater Tunnel has the overflow from the Combined Sewer System as inflow, and its outflow goes straight into the Storage Tower. The outflow only happens, as long as there is space for more water inside the Storage Tower. If there isn't, no water will leave the tunnel. We model the outflow of state $X_{3,t}$ using a flipped Sigmoid function, meaning we are using a positive α. This makes it so that the outflow is "on" until the capacity of the Storage Tower is met. We assume that the Stormwater Tunnel has infinite capacity, and we will therefore not worry about any additional outflow from this state.

The fourth and final state, $X_{4,t}$, will be the Storage Tower. This state has the outflow from $X_{3,t}$ as inflow and has the input P_t as its outflow. This way water can only leave the Storage Tower through the pumping system. To prevent pumping when the Storage Tower is empty, P_t is multiplied by $X_{4,t}$ and a constant, linking pump flow to the current state level.

The model consists of the following system of differential equations:

$$dX_{1,t} = A \cdot U_t dt - \frac{3}{K_1} \cdot X_{1,t} dt + \sigma \cdot dw_{1,t} \tag{11}$$

$$dX_{2,t} = \frac{3}{K_1} \cdot X_{1,t} dt - \frac{3}{K_2} \cdot X_{2,t} dt - \frac{3}{K_{overflow}} \cdot X_{2,t} \cdot \frac{1}{1 + e^{-\alpha_1 (X_{2,t} - \beta_1)}} dt + \sigma \cdot dw_{2,t} \tag{12}$$

$$dX_{3,t} = \frac{3}{K_{overflow}} \cdot X_{2,t} \cdot \frac{1}{1 + e^{-\alpha_1 (X_{2,t} - \beta_1)}} dt - \frac{3}{K_{tunnel}} \cdot X_{3,t} \cdot \frac{1}{1 + e^{\alpha_2 (X_{3,t} - \beta_2)}} dt + \sigma \cdot dw_{3,t} \tag{13}$$

$$dX_{4,t} = \frac{3}{K_{tunnel}} \cdot X_{3,t} \cdot \frac{1}{1 + e^{\alpha_2 (X_{3,t} - \beta_2)}} dt - c_f \cdot P_t dt \cdot X_{4,t} + \sigma \cdot dw_{4,t} \tag{14}$$

Question 2.3.3

We implement the system described in question 2.3.2 using cstmTMB in R, using data from event 1. Since the model's time unit is in hours, the minutely pump flow is multiplied by 60. All variables are scaled by $\frac{1}{1000}$ to simplify parameter estimation, as the volume data is significantly larger compared to pump flow and rainfall (see for example Figure 13).

The estimated parameters for the fitted model are shown in Table 5. Without detailing each parameter, we observe that runoff from state 2 to downstream is noticeably slower than the runoff between states 1 and 2, which explains why overflow occurs in each event. Additionally, β_2 is estimated below 29, which is expected since the scaled maximum capacity of state 4 is 29.[3]

The simulated values with the k-step-ahead predictions set to 10^6 is shown in Figure 11 together with the observed data. The model does not fit the system perfectly, particularly in capturing how the pump flow controls the outflow from state 4. When the volume decreases, the model consistently overestimates the state, and at one point, it stops decreasing even though the state is not empty. This issue is likely

[3]Note that the sigmoid function is equal to 0.5 when $X_{.,t} = \beta$

Parameters	Estimate	Variance	Initial	Lower	Upper
$K1$	3.17	0.51	10	0.1	100
$K2$	7.10	-0.08	10	0.1	100
$K_{overflow}$	10.46	-1.20	10	0.1	100
K_{tunnel}	9.01	-0.07	10	0.1	100
A	7.04	-0.13	10	1e-5	200
α_1	8.22	-218	5	0	100
α_2	13.76	-8.19	5	0	100
β_1	12.79	-4.92	5	0	100
β_2	16.38	-1.47	15	0	100
c_f	0.05	3.09e-5	1.3	0	100

Table 5: Estimated parameters for the model formulated in question 2.3.2

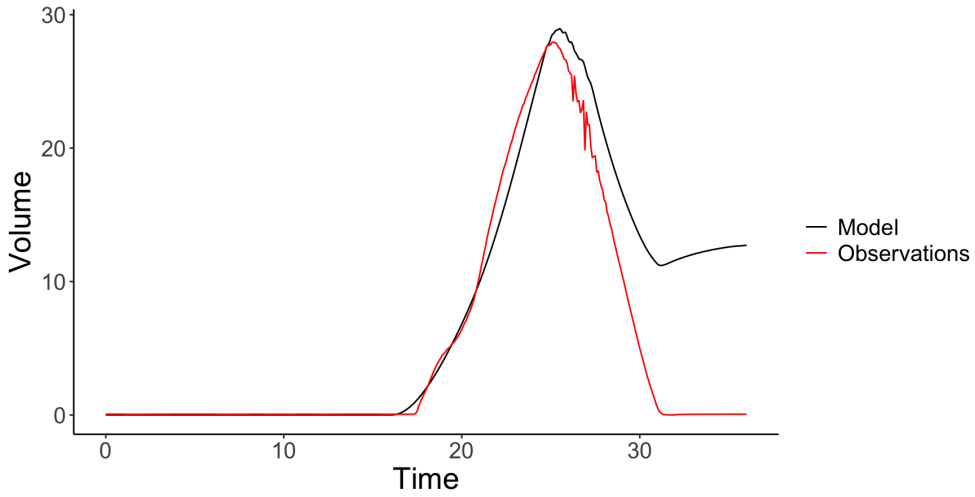


Figure 11: Observed data from event 1 together with the predicted values from the model formulated in question 2.3.2

because the pump flow is linked to the current state volume, so when the volume decreases, so does the pump flow. Additionally, the estimated coefficient c_f is very small, which may contribute to the overestimation. The *nll* for the model is 489.6, but during parameter estimation, we observe that the maximum gradient component is 3500, indicating that the objective function is finding a minimum at a point where there is still a slope present in some direction.

Question 2.3.4.

We now fit the model to the entire dataset. Lacking the method of being able to fit the model to the entire dataset at once we will take an alternative approach. We will fit the model to the events one by one and estimate parameters for each event. Afterwards we take the average of the parameters from the six fitted models and use these to them to fit the model to the entire dataset. The initial values as well as the lower and upper bands are the same as in the previous question, where we only fitted a model to event 1. In order to get the model to converge for each event, we had to increase the tolerance and reduce the stepsize. The estimated parameters are shown in Table 6 The average of all the parameters are displayed in the rightmost column. These are the parameters we will use when fitting the model to each event.

Parameters	Event 1	Event 2	Event 3	Event 4	Event 5	Event 6	Average
$K1$	3.17	10.19	9.11	3.90	9.53	10.04	7.66
$K2$	7.10	9.85	8.67	4.46	7.48	9.88	7.90
$K_{overflow}$	10.46	10.28	9.15	6.22	10.58	10.13	9.47
K_{tunnel}	9.01	9.40	2.30	5.40	9.40	9.55	7.51
A	7.04	9.61	5.81	6.58	5.91	9.73	7.45
α_1	8.22	5.00	5.68	5.76	5.10	5.00	5.79
α_2	13.76	4.75	4.90	4.63	5.00	4.64	6.28
β_1	12.79	4.98	8.42	9.70	6.56	4.97	7.90
β_2	16.38	15.60	16.60	11.38	15.00	15.29	15.04
c_f	0.05	0.17	0.18	0.28	0.41	0.07	0.19
$log(\sigma)$	0.00	0.05	0.00	0.00	0.00	0.00	0.00
$log(\sigma_Y)$	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Table 6: Estimated parameters for all events and the average.

On the plots shown in Figure 12 we have plotted the observed data together with the fit of the model. Even though the model, generally, isn't a very good fit of the data, it does have some strengths. It is able to capture the initial increase in volume, that happens due to the rainfall and the delay caused by the overflow into the Stormwater Tunnel, which aligns with the physical behavior of the system.

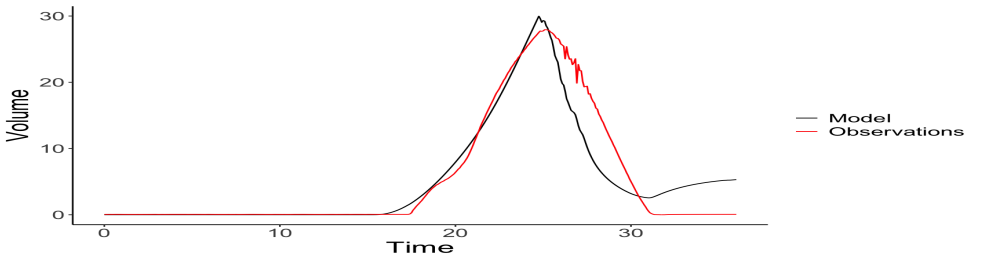
One of the biggest problems with the model is that it consistently overestimates the volume inside the Storage Tower during the phase, where the pump is active. This leads to inaccuracies, when emptying the Storage Tower. This occurs because the pump flow is multiplied by $X_{4,t}$ and is thus directly tied to the Storage Tower volume, which results in weaker pumping rates, as the volume inside the tower declines. Even though the volume is supposed to become zero, the model fails to do so, which indicates, that the pumpflow isn't modeled correctly.

Apart from pumping phase, the model seems to underestimate the volume in the Storage Tower. This is likely because the flipped Sigmoid function isn't able to capture all the nuances of the system dynamics, and therefore contributes to some inaccuracies.

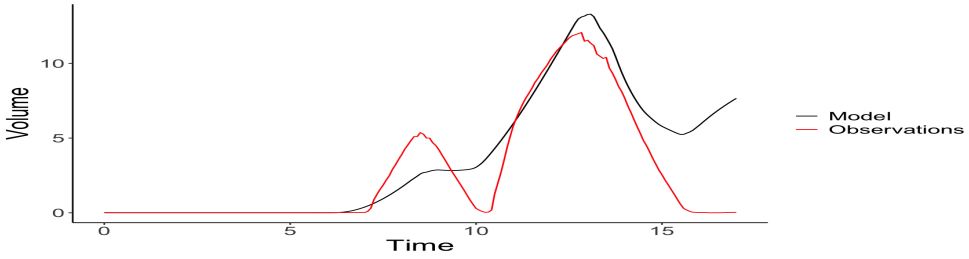
Certain events are harder for the model to predict i.e. the case in event 2 with two peaks, we see that the model smooths out the two peaks into one.

Question 2.3.5

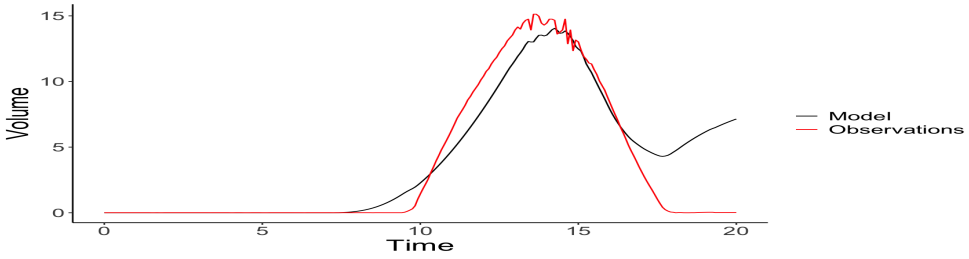
We have seen that the formulated model has difficulty replicating the dynamics of stormwater volume in the storage tower when it comes to the storage being emptied. We have also discussed how this was probably related to the fact that we have multiplied the pump flow discharge with the state $X_{4,t}$. We could have used a non-linear formulation to model the pump flow in a way so that it is not pumping when the storage tower is empty. For instance, using a similar sigmoid function to model pump activation/deactivation could be one way to go. If additional data were available about the specific pumping operation as control thresholds, on-off schedules, or pumping rates based on water levels, then we could specify more states for the pumping system to more accurately represent its behavior. Even general information about rule-based systems for pump flow schedules would be beneficial. Lastly, we could extend the model by splitting certain states into multiple states to find the most suitable level of detail, as we did in part 2.1. For instance, instead of assuming the stormwater tunnel has unlimited capacity,



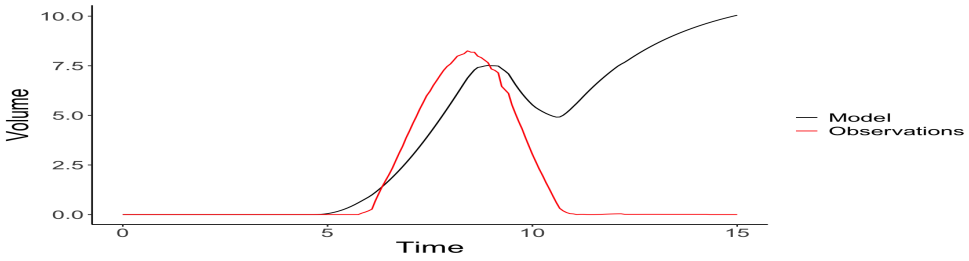
(a) Event 1 with t_0 = 2018-08-12 04:00:00



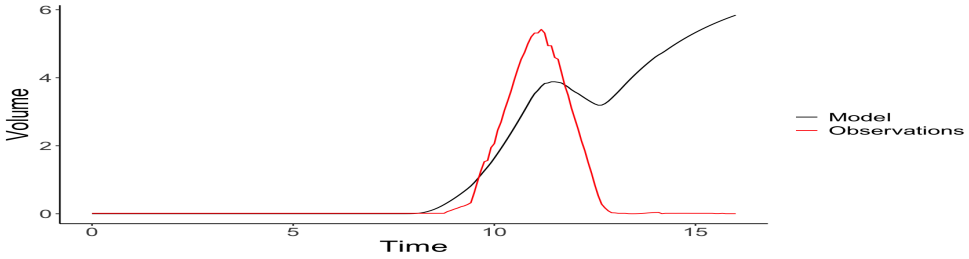
(b) Event 2 with t_0 = 2018-08-27 07:00:00



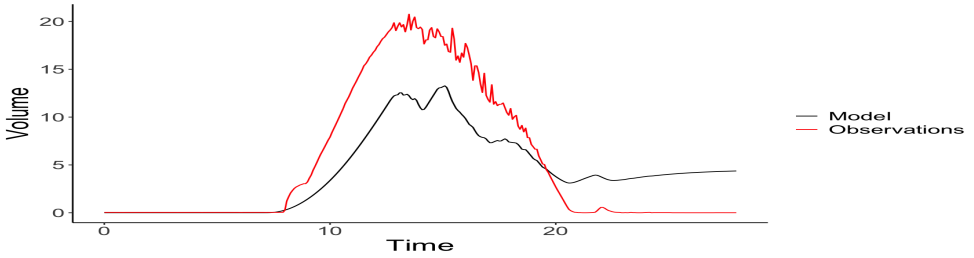
(c) Event 3 with t_0 = 2018-08-30 04:00:00



(d) Event 4 with t_0 = 2018-10-06 21:00:00



(e) Event 5 with t_0 = 2018-11-11 00:00:00



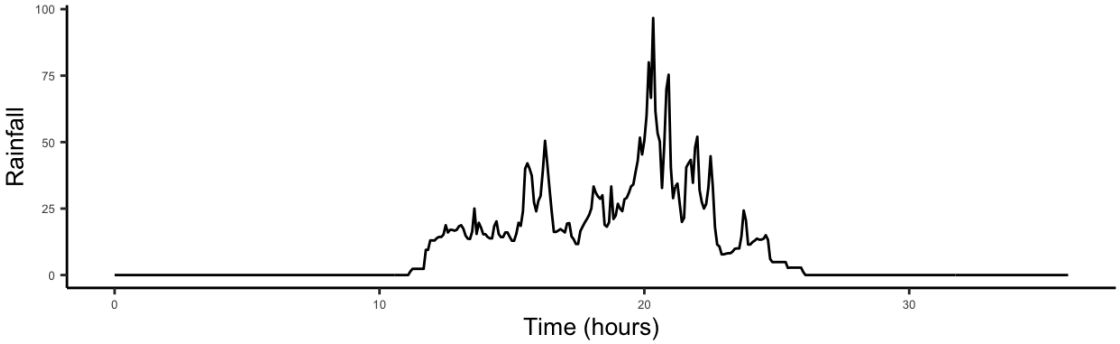
(f) Event 6 with t_0 = 2018-12-21 12:00:00

Figure 12: Observations together with model fit for all 6 events

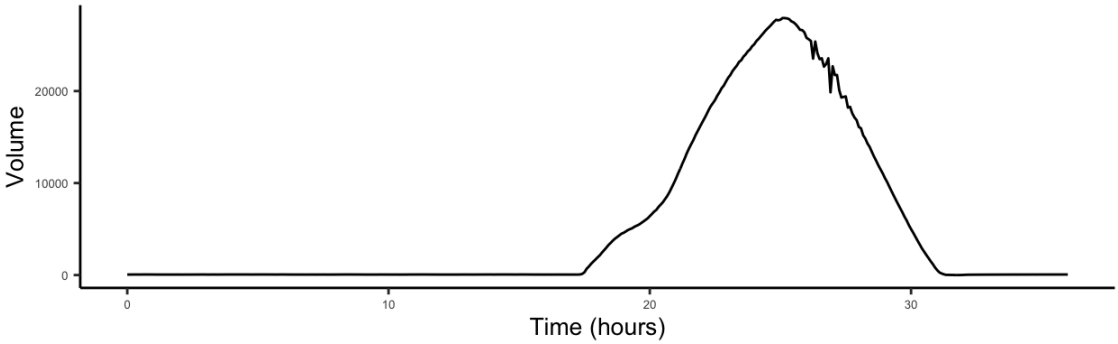
we could model it with multiple states and see which model has the lowest nll. However, this is related to the inflow to state 4, which is modeled quite well.

3 Appendex A

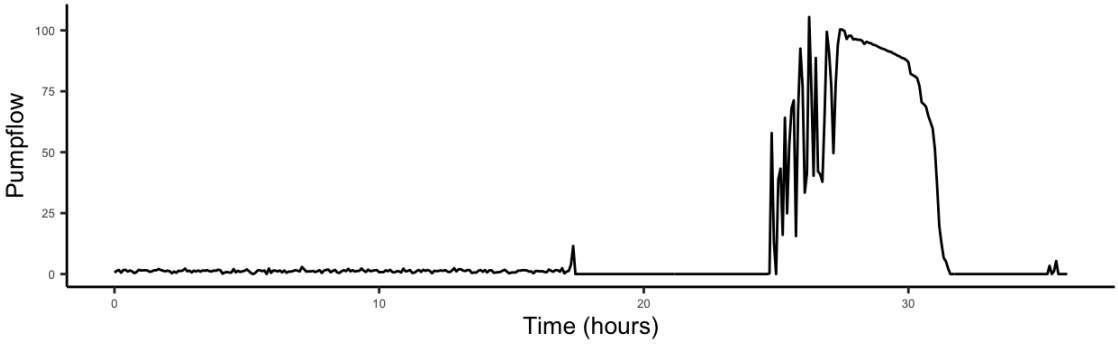
3.1 Plot of observations from the larger real life rainfall-runoff case



(a) Rainfall for event 1 with t_0 = 2018-08-12 04:00:00

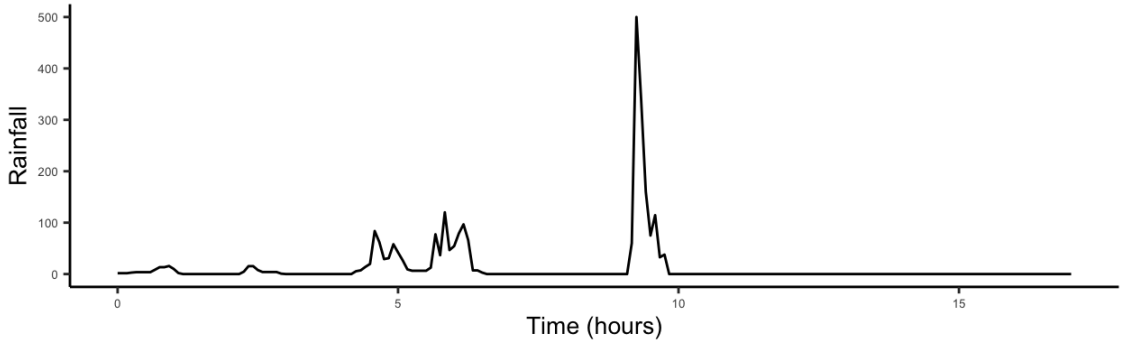


(b) Stormwater volume for event 1 with t_0 = 2018-08-12 04:00:00

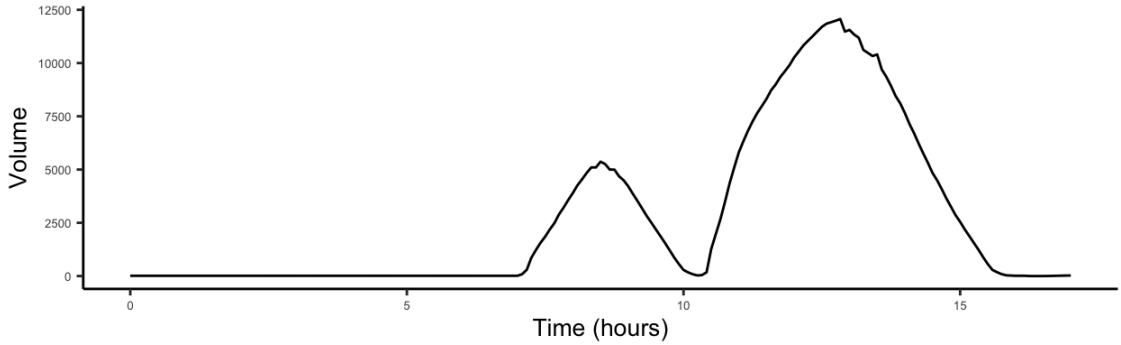


(c) Pumpflow for event 1 with t_0 = 2018-08-12 04:00:00

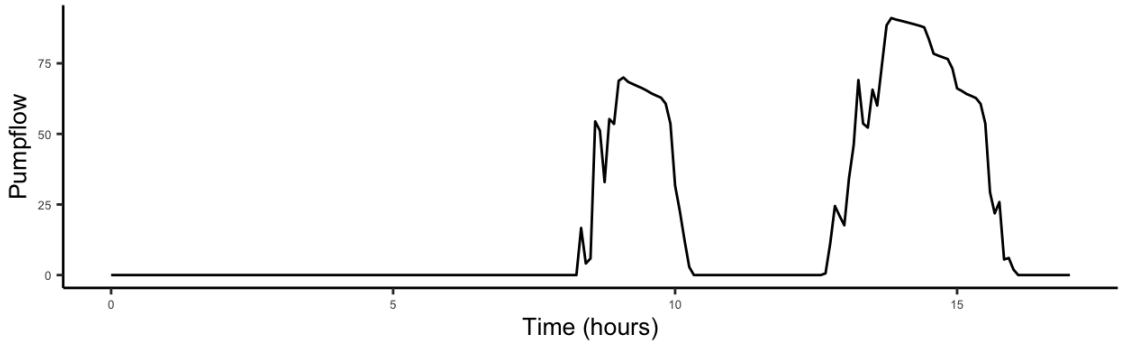
Figure 13: Observations for event 1



(a) Rainfall for event 2 with t_0 = 2018-08-27 07:00:00

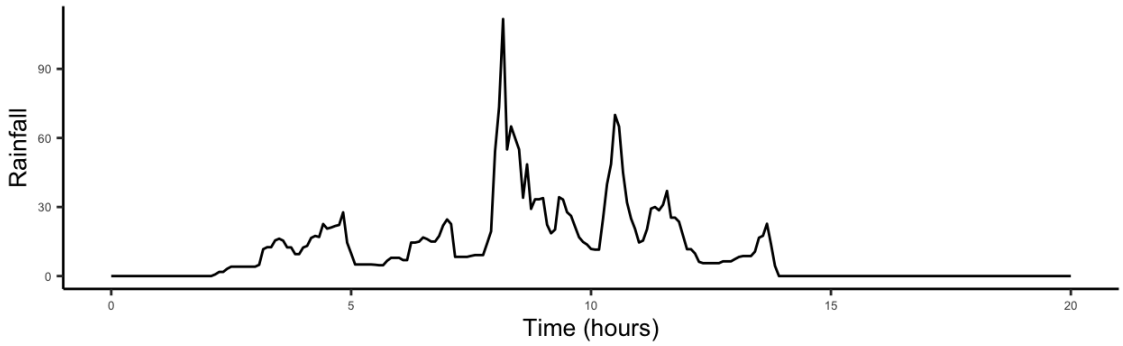


(b) Stormwater volume for event 2 with t_0 = 2018-08-27 07:00:00

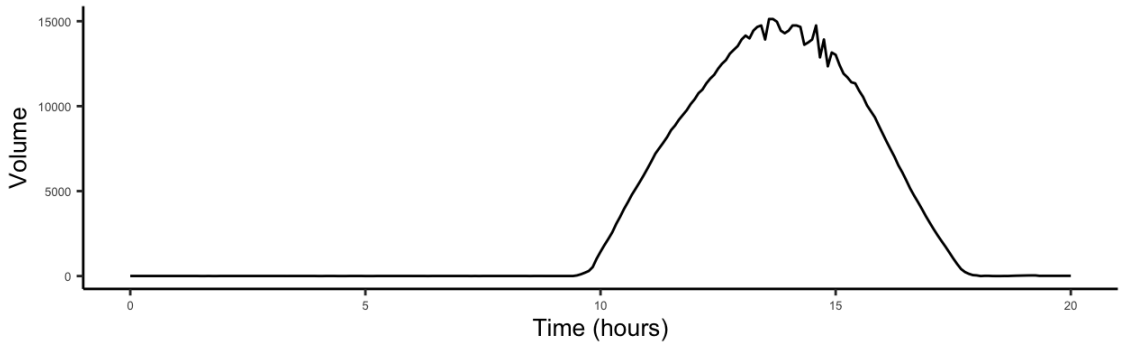


(c) Pumpflow for event 2 with t_0 = 2018-08-27 07:00:00

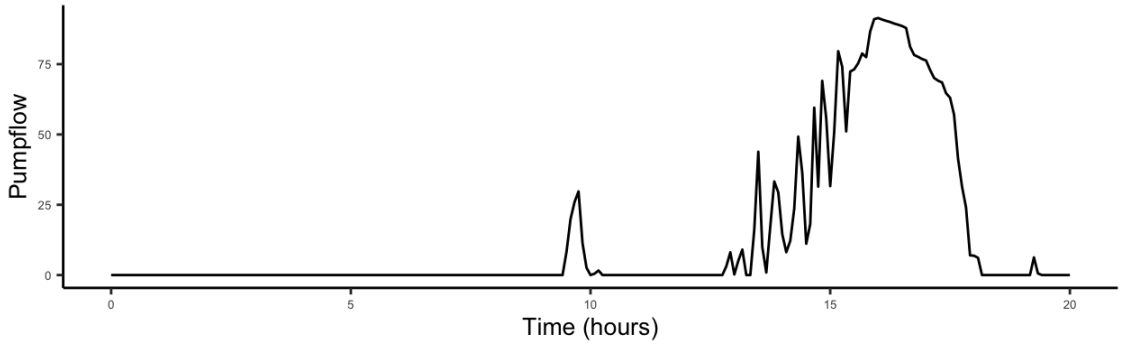
Figure 14: Observations for event 2



(a) Rainfall for event 3 with t_0 = 2018-08-30 04:00:00

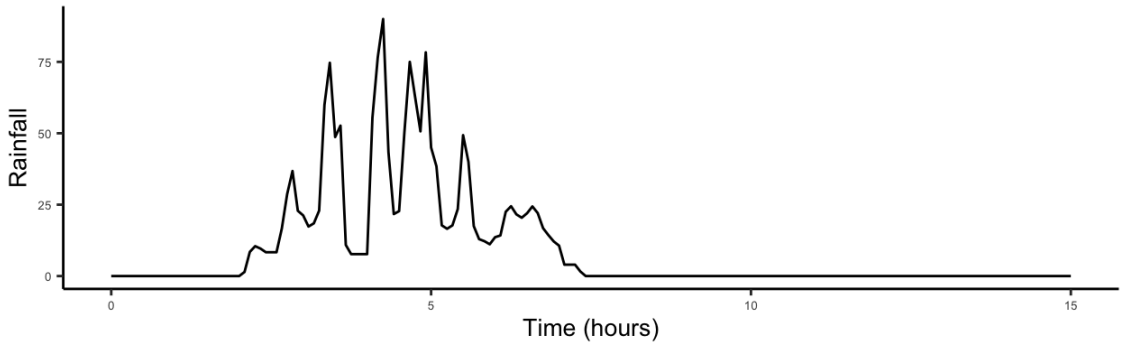


(b) Stormwater volume for event 3 with t_0 = 2018-08-30 04:00:00

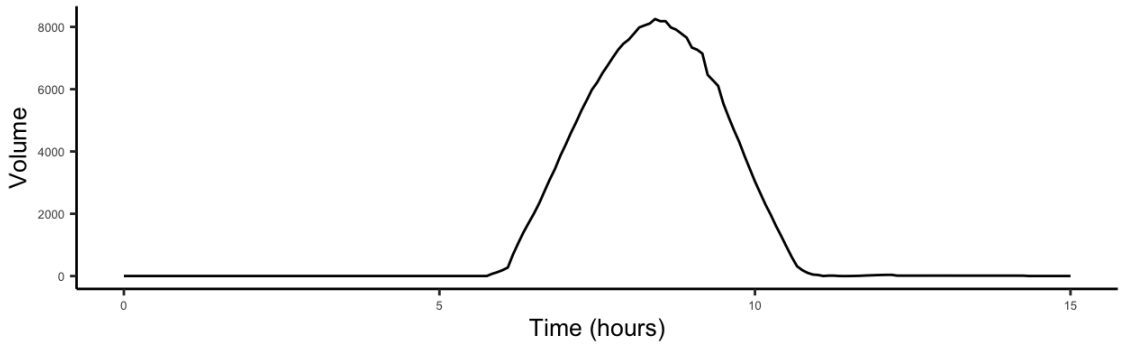


(c) Pumpflow for event 3 with t_0 = 2018-08-30 04:00:00

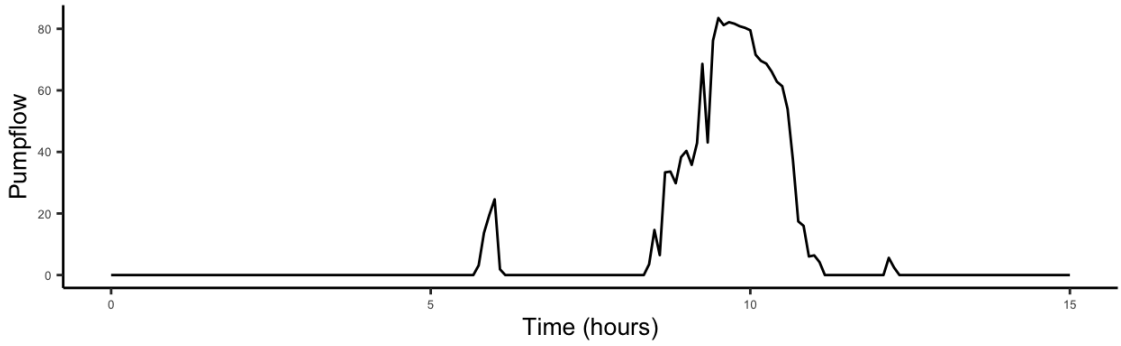
Figure 15: Observations for event 3



(a) Rainfall for event 4 with t_0 = 2018-10-06 21:00:00

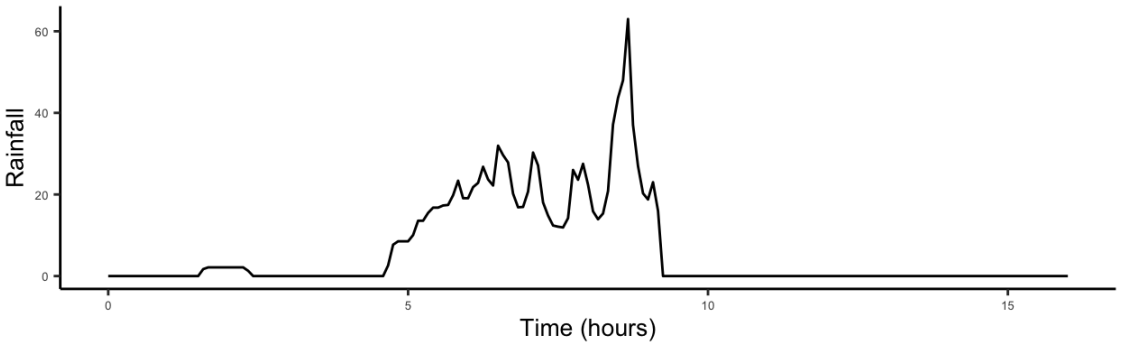


(b) Stormwater volume for event 4 with t_0 = 2018-10-06 21:00:00

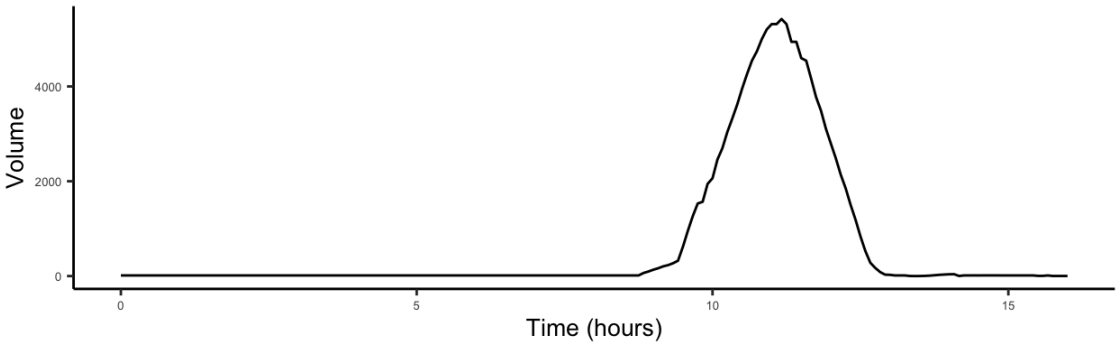


(c) Pumpflow for event 4 with t_0 = 2018-10-06 21:00:00

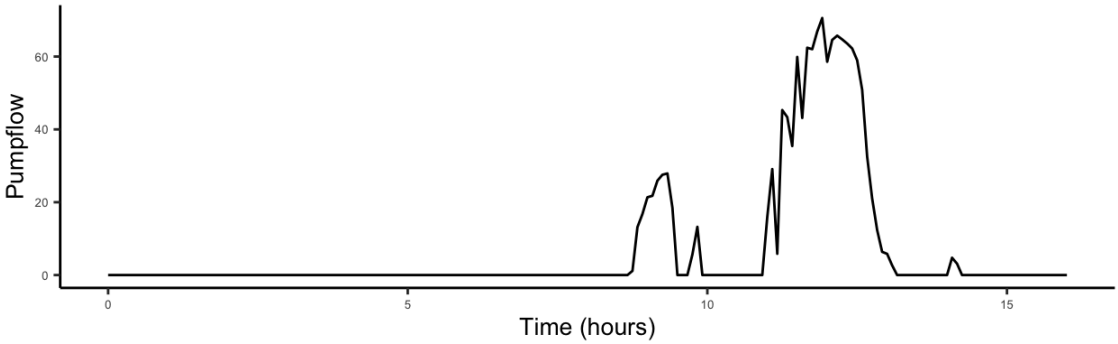
Figure 16: Observations for event 4



(a) Rainfall for event 5 with t_0 = 2018-11-11 00:00:00

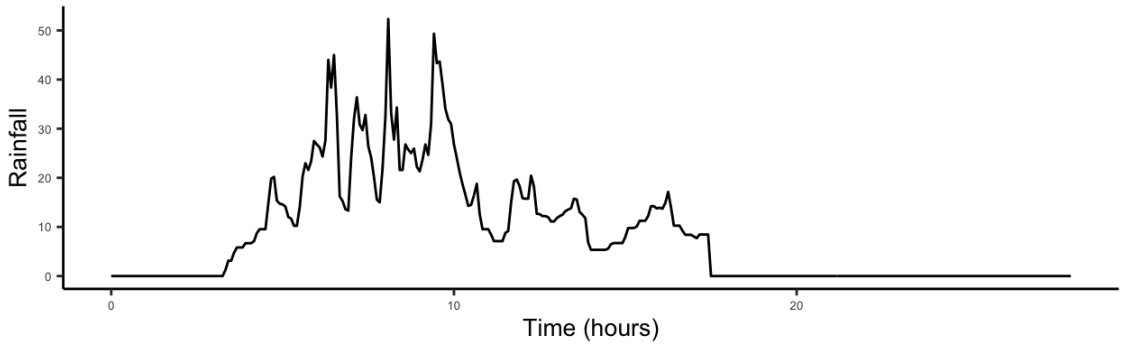


(b) Stormwater volume for event 5 with t_0 = 2018-11-11 00:00:00

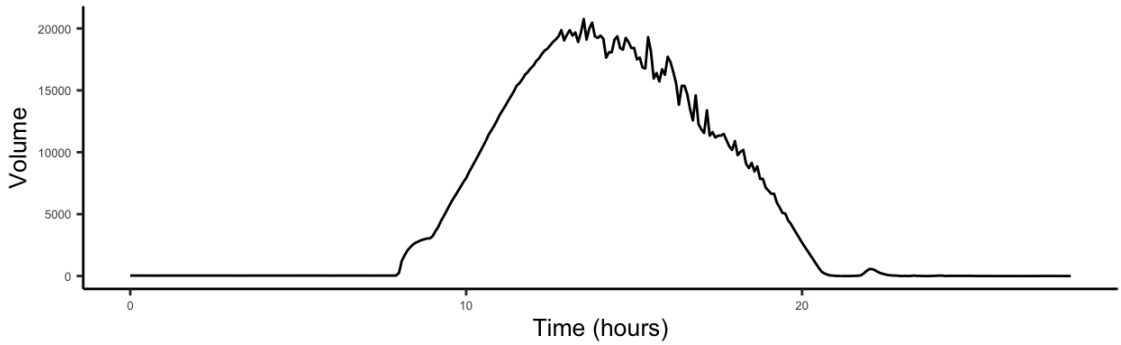


(c) Pumpflow for event 5 with t_0 = 2018-11-11 00:00:00

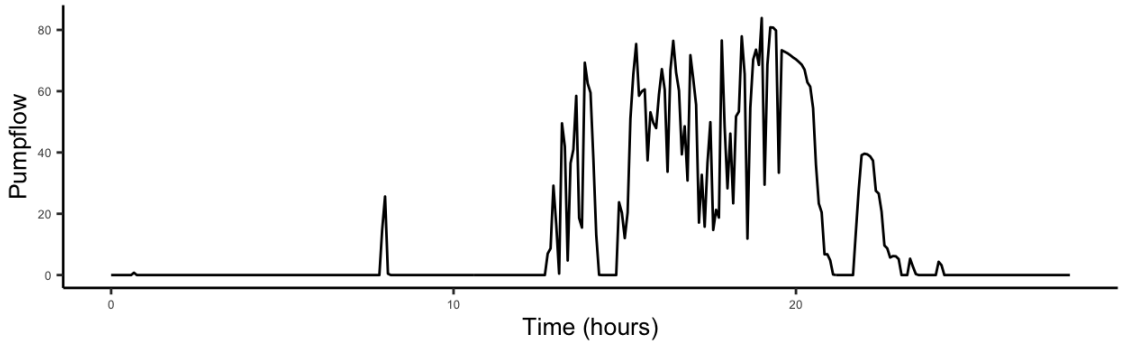
Figure 17: Observations for event 5



(a) Rainfall for event 6 with t_0 = 2018-12-21 12:00:00



(b) Stormwater volume for event 6 with t_0 = 2018-12-21 12:00:00



(c) Pumpflow for event 6 with t_0 = 2018-12-21 12:00:00

Figure 18: Observations for event 6

3.2 Code

```
library(ctsmTMB)
library(ggplot2)
library(dplyr)

rm(list = ls())
dev.off()

################################################################################
# Part 1 question 1.1
################################################################################
```

```
delta <- 2^(-9)
time_interval <- seq(0,100, delta)
n <- length(time_interval)

Y1 <- matrix(data = NA, nrow = n, ncol = 5)
Y2 <- matrix(data = NA, nrow = n, ncol = 5)



Y1[1,] <- -1.9
Y2[1,] <- 1.2

theta_1 <- 0.7
theta_2 <- 0.8
theta_3 <- 3.0
theta_4 <- -0.34
sigma <- c(0, 0.1, 0.2, 0.3, 0.4)

set.seed(123)
W <- rnorm(n, mean = 0, sd = sqrt(delta))


for (i in 1:length(sigma)){
  for(j in 2:n){

    Y1[j,i] <- Y1[j-1,i] + theta_3*(Y1[j-1,i] + Y2[j-1,i] - 1/3*Y1[j-1,i]^3 + theta_4)*delta
        + sigma[i]*W[j]
    Y2[j,i] <- Y2[j-1,i] - 1/theta_3*(Y1[j-1,i] + theta_2*Y2[j-1,i] - theta_1)*delta

  }
}

Y1 <- data.frame(Y1)
Y2 <- data.frame(Y2)

p_test <- ggplot() +
  geom_line(data = Y1, aes(x = time_interval, y = X1, color = "X1")) +
  geom_line(data = Y1, aes(x = time_interval, y = X2, color = "X2")) +
  geom_line(data = Y1, aes(x = time_interval, y = X3, color = "X3")) +
  xlab("Time") +
  ylab("Y1") +
  scale_color_manual(values = c("X1" = "black", "X2" = "orange", "X3" = "blue4")) + #
      Specify custom colors for each line
  theme_classic() +
  theme(axis.title = element_text(size = 18))

p_test

# Plot realizations for all 4 sigma ------------------------------------------
Y1_s0_plot <- ggplot(data = Y1, aes(x = time_interval, y = X1)) +
  geom_line() +
  xlab( "Time") +
```

```
  ylab("Y1") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y1_s0_plot

Y2_s0_plot <- ggplot(data = Y2, aes(x = time_interval, y = X1)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y2_s0_plot

Y1_s1_plot <- ggplot(data = Y1, aes(x = time_interval, y = X2)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y1") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y1_s1_plot

Y2_s1_plot <- ggplot(data = Y2, aes(x = time_interval, y = X2)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y2_s1_plot

Y1_s2_plot <- ggplot(data = Y1, aes(x = time_interval, y = X3)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y1") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y1_s2_plot

Y2_s2_plot <- ggplot(data = Y2, aes(x = time_interval, y = X3)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y2_s2_plot

Y1_s3_plot <- ggplot(data = Y1, aes(x = time_interval, y = X4)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y1") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y1_s3_plot
```

```
Y2_s3_plot <- ggplot(data = Y2, aes(x = time_interval, y = X4)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y2_s3_plot

Y1_s4_plot <- ggplot(data = Y1, aes(x = time_interval, y = X5)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y1") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y1_s4_plot

Y2_s4_plot <- ggplot(data = Y2, aes(x = time_interval, y = X5)) +
  geom_line() +
  xlab( "Time") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y2_s4_plot


# Plot (Y1, Y2) for all sigma ----------------------------------------------------

Y <- data.frame(Y1,Y2)

Y_s0_plot <- ggplot(data = Y, aes(x = X1, y = X1.1)) +
  geom_path() + # Use geom_path instead of geom_line
  xlab("Y1") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y_s0_plot

Y_s1_plot <- ggplot(data = Y, aes(x = X2, y = X2.1)) +
  geom_path() + # Use geom_path instead of geom_line
  xlab("Y1") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y_s1_plot

Y_s2_plot <- ggplot(data = Y, aes(x = X3, y = X3.1)) +
  geom_path() + # Use geom_path instead of geom_line
  xlab("Y1") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
```

```
Y_s2_plot

Y_s3_plot <- ggplot(data = Y, aes(x = X4, y = X4.1)) +
  geom_path() + # Use geom_path instead of geom_line
  xlab("Y1") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y_s3_plot

Y_s4_plot <- ggplot(data = Y, aes(x = X5, y = X5.1)) +
  geom_path() + # Use geom_path instead of geom_line
  xlab("Y1") +
  ylab("Y2") +
  theme_classic() +
  theme(axis.title = element_text(size = 18))
Y_s4_plot


################################################################################
# Part 1 question 1.2
################################################################################

# Plot 2D histogram ------------------------------------------------------------

# sigma = 0.1
p1 <- ggplot(data=Y, aes(x = X2, y = X2.1)) +
  geom_bin2d(bins = 100) + # Partitioning into 100x100 cells
  scale_fill_gradientn(colours = c("blue", "lightblue", "yellow", "orange", "red"),
                       values = scales::rescale(c(0, 1, 10, 50, max(table(cut(Y$X2, 100),
                           cut(Y$X2.1, 100))))),
                       na.value = "blue") +
  theme_classic() +
  labs(x = "Y1",y = "Y2", fill = "C") +
  theme(axis.title = element_text(size = 18))
p1

# sigma = 0.2
p2 <- ggplot(data=Y, aes(x = X3, y = X3.1)) +
  geom_bin2d(bins = 100) + # Partitioning into 100x100 cells
  scale_fill_gradientn(colours = c("blue", "lightblue", "yellow", "orange", "red"),
                       values = scales::rescale(c(0, 1, 10, 50, max(table(cut(Y$X3, 100),
                           cut(Y$X3.1, 100))))),
                       na.value = "blue") +
  theme_classic() +
  labs(x = "Y1",y = "Y2", fill = "C") +
  theme(axis.title = element_text(size = 18))
p2

# sigma = 0.3
p3 <- ggplot(data=Y, aes(x = X4, y = X4.1)) +
  geom_bin2d(bins = 100) + # Partitioning into 100x100 cells
```

```
  scale_fill_gradientn(colours = c("blue", "lightblue", "yellow", "orange", "red"),
                       values = scales::rescale(c(0, 1, 10, 50, max(table(cut(Y$X4, 100),
                           cut(Y$X4.1, 100))))),
                       na.value = "blue") +
  theme_classic() +
  labs(x = "Y1",y = "Y2", fill = "C") +
  theme(axis.title = element_text(size = 18))
p3

# sigma = 0.4
p4 <- ggplot(data=Y, aes(x = X5, y = X5.1)) +
  geom_bin2d(bins = 100) + # Partitioning into 100x100 cells
  scale_fill_gradientn(colours = c("blue", "lightblue", "yellow", "orange", "red"),
                       values = scales::rescale(c(0, 1, 10, 50, max(table(cut(Y$X5, 100),
                           cut(Y$X5.1, 100))))),
                       na.value = "blue") +
  theme_classic() +
  labs(x = "Y1",y = "Y2", fill = "C") +
  theme(axis.title = element_text(size = 18))
p4



################################################################################
# Part 2.1
################################################################################

rm(list = ls())
dev.off()

# Read data
data <- read.csv("ex1_rainfallrunoff copy.csv", sep = ",")

# Create a new data frame with the necessary variables
.data <- data.frame(
  t = seq(0, 10, length.out = nrow(data)),
  #t = seq_len(nrow(data)),
  rainfall = as.numeric(data$rainfall),
  Y = as.numeric(data$stormwater)
)

str(.data)


# Create different model object ------------------------------------------------

model1 = ctsmTMB$new()
model2 = ctsmTMB$new()
model3 = ctsmTMB$new()
model4 = ctsmTMB$new()
model5 = ctsmTMB$new()
```

```
# name of models and the created .cpp file ----------------------------------
model1$setModelname("Linear_rainfall_runoff_2_states")
model2$setModelname("Linear_rainfall_runoff_3_states")
model3$setModelname("Linear_rainfall_runoff_4_states")
model4$setModelname("Linear_rainfall_runoff_5_states")
model5$setModelname("Linear_rainfall_runoff_6_states")

# System equations ----------------------------------------------------------
model1$addSystem(
  dx1 ~ (A * rainfall - (1 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (1 / K) * x1 * dt + sigma * dw2
)

model2$addSystem(
  dx1 ~ (A * rainfall - (2 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (2/K * x1 - (2 / K) * x2) * dt + sigma * dw2,
  dx3 ~ (2 / K) * x2 * dt + sigma * dw3
)

model3$addSystem(
  dx1 ~ (A * rainfall - (3 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (3/K * x1 - (3 / K) * x2) * dt + sigma * dw2,
  dx3 ~ (3/K * x2 - (3 / K) * x3) * dt + sigma * dw3,
  dx4 ~ (3 / K) * x3 * dt + sigma * dw4
)

model4$addSystem(
  dx1 ~ (A * rainfall - (4 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (4/K * x1 - (4 / K) * x2) * dt + sigma * dw2,
  dx3 ~ (4/K * x2 - (4 / K) * x3) * dt + sigma * dw3,
  dx4 ~ (4/K * x3 - (4 / K) * x4) * dt + sigma * dw4,
  dx5 ~ (4 / K) * x4 * dt + sigma * dw5
)

model5$addSystem(
  dx1 ~ (A * rainfall - (5 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (5/K * x1 - (5 / K) * x2) * dt + sigma * dw2,
  dx3 ~ (5/K * x2 - (5 / K) * x3) * dt + sigma * dw3,
  dx4 ~ (5/K * x3 - (5 / K) * x4) * dt + sigma * dw4,
  dx5 ~ (5/K * x4 - (5 / K) * x5) * dt + sigma * dw5,
  dx6 ~ (5 / K) * x5 * dt + sigma * dw6
)


# Observation equations -----------------------------------------------------
model1$addObs(
  Y ~ x2
)

model2$addObs(
  Y ~ x3
)
```

```
model3$addObs(
  Y ~ x4
)

model4$addObs(
  Y ~ x5
)

model5$addObs(
  Y ~ x6
)


# Set observation equation variances ------------------------------------------
model1$setVariance(
  Y ~ sigma_Y^2
)
model2$setVariance(
  Y ~ sigma_Y^2
)
model3$setVariance(
  Y ~ sigma_Y^2
)
model4$setVariance(
  Y ~ sigma_Y^2
)
model5$setVariance(
  Y ~ sigma_Y^2
)

# Specify algebraic relations -------------------------------------------------
model1$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)

model2$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)

model3$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)

model4$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)
```

```
model5$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)

# vector input ------------------------------------------------------------
model1$addInput(rainfall)
model2$addInput(rainfall)
model3$addInput(rainfall)
model4$addInput(rainfall)
model5$addInput(rainfall)




# Specify parameter initial values and lower/upper bounds in estimation --------
model1$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)

model2$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)

model3$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)

model4$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)

model5$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)
```

```
# Set initial state mean and covariance ---------------------------------------
model1$setInitialState(list(mean = c(0,0), 0.1*diag(2)))
model2$setInitialState(list(mean = c(0,0,0), 0.1*diag(3)))
model3$setInitialState(list(mean = c(0,0,0,0), 0.1*diag(4)))
model4$setInitialState(list(mean = c(0,0,0,0,0), 0.1*diag(5)))
model5$setInitialState(list(mean = c(0,0,0,0,0,0), 0.1*diag(6)))


# Carry out estimations with default settings (extended Kalman filter) ---------
fit1 <- model1$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)

fit2 <- model2$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)

fit3 <- model3$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)

fit4 <- model4$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)

fit5 <- model5$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)

fit1$par.fixed

# 2.1.2 - Compare models ------------------------------------------------------

fit1$nll
fit2$nll
fit3$nll
fit4$nll
fit5$nll

nll <- c(fit1$nll, fit2$nll, fit3$nll, fit4$nll, fit5$nll)
nll_data <- data.frame(nll = nll,
```

```
                    states = c(2,3,4,5,6))
p_nll <- ggplot(data = nll_data, aes(x = states, y = nll)) +
  geom_line() +
  theme_classic() +
  labs(x = "States",y = "nll") +
  theme(axis.title = element_text(size = 18))

p_nll
# fit 3 is best, 4 states

# 2.1.3 - Parameters ------------------------------------------------------

fit1$par.fixed # parameters with 2 staes
fit1$cov.fixed
fit3$par.fixed # parameters in final model.
fit3$cov.fixed
cov2cor(fit3$cov.fixed)

# A and K is smaller.

plot(fit1)
plot(fit3)

# 2.1.4 - correlation matrix ----------------------------------------------

cov2cor(fit3$cov.fixed)

# there is a 0.4 (i.e somewhat strong) correslation between the area within
# which the rainfall is relevant and the time scale of the runoff process as
# expected

# 2.1.5 - model validation ------------------------------------------------

plot(fit3)
plot(fit1)
# autocorealtion looks like white noice
# QQ plot reveals it is realtive normal
# residuals looks nicely spread.


pred.list3 = model3$predict(data=.data,
                            k.ahead=1e6,
                            method="ekf",
)




ggplot() +
  geom_line(data=pred.list3$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
```

```
  labs(x="Time", y="Runoff", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
```

```
################################################################################
# Part 2.2
################################################################################

library(ctsmTMB)
rm(list = ls())
dev.off()

# Read data
data <- read.csv("ex2_overflow.csv", sep = ",")

# Create a new data frame with the necessary variables
.data <- data.frame(
  t = seq(0, 10, length.out = nrow(data)),
  rainfall = as.numeric(data$rainfall),
  Y = as.numeric(data$stormwater)
)

str(.data)


# Create model object
model = ctsmTMB$new()
model1 = ctsmTMB$new()

# Set name of model (and the created .cpp file)
model$setModelname("Linear_rainfall_overflow_3_states")
model1$setModelname("Non_Linear_rainfall_overflow_3_states")

# Add system equations
model$addSystem(
  dx1 ~ (A * rainfall - (2 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (2/K * x1 - (2/K)*x2 - (2/K_overflow)*(x2 - b)) * dt + sigma * dw2,
  dx3 ~ (2 / K_overflow) * (x2 - b) * dt + sigma * dw3
)
```

```
model1$addSystem(
  dx1 ~ (A * rainfall - (2 / K) * x1) * dt + sigma * dw1,
  dx2 ~ (2/K * x1 - (2/K)*x2 - (2/K_overflow)*(x2*(1/(1+exp(-a*(x2-b)))))) * dt + sigma *
      dw2,
  dx3 ~ (2/K_overflow)*(x2*(1/(1+exp(-a*(x2-b)))))* dt + sigma * dw3
)

# Add observation equation
model$addObs(
  Y ~ x3
)
model1$addObs(
  Y ~ x3
)


model$setVariance(
  Y ~ sigma_Y^2
)
model1$setVariance(
  Y ~ sigma_Y^2
)


model$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)


model1$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)



# Specify parameter initial values and lower/upper bounds in estimation --------
model$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  K_overflow = c(initial = 1, lower = 1e-10, upper = 50),
  b = c(initial = 4, lower = 0, upper = 10),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)
model1$setParameter(
  A       = c(initial = 1, lower = 1e-5, upper = 50),
  logsigma  = log(c(initial = 1, lower = 1e-10, upper = 30)),
  K       = c(initial = 1, lower = 1e-10, upper = 50),
  b = c(initial = 0.5, lower = 0, upper = 50),
```

```
  a = c(initial = 1, lower = 0, upper = 50),
  K_overflow = c(initial = 1, lower = 1e-10, upper = 50),
  logsigma_Y = log(c(initial = 1, lower = 1e-10, upper = 30))
)


# Add vector input
model$addInput(rainfall)
model1$addInput(rainfall)


# Set initial state mean and covariance
model$setInitialState(list(mean = c(0,0,0), 0.1*diag(3)))
model1$setInitialState(list(mean = c(0,0,0), 0.1*diag(3)))

# Carry out estimation with default settings (extended Kalman filter)
fit <- model$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)
fit1 <- model1$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)

plot(fit)
plot(fit1)

fit$par.fixed
fit1$par.fixed
fit1$cov.fixed

fit$cov.fixed
cov2cor(fit$cov.fixed)
fit1$cov.fixed
cov2cor(fit1$cov.fixed)


# Perform full prediction without data update
pred.list = model$predict(data=.data,
                          k.ahead=1e6,
                          method="ekf",
)
pred.list1 = model1$predict(data=.data,
                        k.ahead=1e6,
                        method="ekf",
)

ggplot() +
  geom_line(data=pred.list$observations, aes(x=t.j, y=Y, color="Model")) +
```

```
  geom_line(data = .data, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Runoff", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )


ggplot() +
  geom_line(data=pred.list1$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Runoff", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
```

```
################################################################################
# Part 2.3
################################################################################
library(lubridate)


rm(list = ls())
dev.off()

# Read data
data <- read.csv("ex3_largecase.csv", sep = ",")



# 2.3.1 - Plot the data ------------------------------------------------------

data$Timestamp <- ymd_hms(data$Timestamp)

df.event1 <- data[data$Event_ID == 1, ]
df.event2 <- data[data$Event_ID == 2, ]
df.event3 <- data[data$Event_ID == 3, ]
```

```
df.event4 <- data[data$Event_ID == 4, ]
df.event5 <- data[data$Event_ID == 5, ]
df.event6 <- data[data$Event_ID == 6, ]


df.event1$t <- as.numeric(difftime(df.event1$Timestamp, df.event1$Timestamp[1], units =
    "hours"))
df.event2$t <- as.numeric(difftime(df.event2$Timestamp, df.event2$Timestamp[1], units =
    "hours"))
df.event3$t <- as.numeric(difftime(df.event3$Timestamp, df.event3$Timestamp[1], units =
    "hours"))
df.event4$t <- as.numeric(difftime(df.event4$Timestamp, df.event4$Timestamp[1], units =
    "hours"))
df.event5$t <- as.numeric(difftime(df.event5$Timestamp, df.event5$Timestamp[1], units =
    "hours"))
df.event6$t <- as.numeric(difftime(df.event6$Timestamp, df.event6$Timestamp[1], units =
    "hours"))

par(mfrow = c(3, 1))
plot(df.event1$t,df.event1$Volume, type = "l")


# Plot the data with ggplot2
p.event1.volume <- ggplot(df.event1, aes(x = t, y = Volume)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Volume") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )
p.event1.pump <- ggplot(df.event1, aes(x = t, y = Pumpflow)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Pumpflow") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event1.rainflow <- ggplot(df.event1, aes(x = t, y = Rainfall)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Rainfall") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
```

```
    legend.text = element_text(size = 5)
  )

p.event1.volume
p.event1.pump
p.event1.rainflow



p.event2.volume <- ggplot(df.event2, aes(x = t, y = Volume)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Volume") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )
p.event2.pump <- ggplot(df.event2, aes(x = t, y = Pumpflow)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Pumpflow") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event2.rainflow <- ggplot(df.event2, aes(x = t, y = Rainfall)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Rainfall") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event2.volume
p.event2.pump
p.event2.rainflow



p.event3.volume <- ggplot(df.event3, aes(x = t, y = Volume)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Volume") +
  theme_classic() +
  theme(
```

```
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )
p.event3.pump <- ggplot(df.event3, aes(x = t, y = Pumpflow)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Pumpflow") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event3.rainflow <- ggplot(df.event3, aes(x = t, y = Rainfall)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Rainfall") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event3.volume
p.event3.pump
p.event3.rainflow



p.event4.volume <- ggplot(df.event4, aes(x = t, y = Volume)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Volume") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )
p.event4.pump <- ggplot(df.event4, aes(x = t, y = Pumpflow)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Pumpflow") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
```

```
  )

p.event4.rainflow <- ggplot(df.event4, aes(x = t, y = Rainfall)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Rainfall") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event4.volume
p.event4.pump
p.event4.rainflow


p.event5.volume <- ggplot(df.event5, aes(x = t, y = Volume)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Volume") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )
p.event5.pump <- ggplot(df.event5, aes(x = t, y = Pumpflow)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Pumpflow") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event5.rainflow <- ggplot(df.event5, aes(x = t, y = Rainfall)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Rainfall") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event5.volume
p.event5.pump
```

```
p.event5.rainflow
```

```
p.event6.volume <- ggplot(df.event6, aes(x = t, y = Volume)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Volume") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )
p.event6.pump <- ggplot(df.event6, aes(x = t, y = Pumpflow)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Pumpflow") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event6.rainflow <- ggplot(df.event6, aes(x = t, y = Rainfall)) +
  geom_line() +
  labs(x = "Time (hours)", y = "Rainfall") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 10),
    axis.text = element_text(size = 5),
    legend.title = element_text(size = 10),
    legend.text = element_text(size = 5)
  )

p.event6.volume
p.event6.pump
p.event6.rainflow
```

```
# 2.3.3 - model system ------------------------------------------------------
```

```
# Create a new data frame with the necessary variables
.data <- data.frame(
  t = df.event1$t,
```

```
  Rainfall = 60*as.numeric(df.event1$Rainfall/1000),
  Pumpflow = 60*as.numeric(df.event1$Pumpflow)/1000,
  Y = as.numeric(df.event1$Volume)/1000
)

str(.data)


# Create model object
model = ctsmTMB$new()


# Set name of model (and the created .cpp file)
model$setModelname("larger_real_life_case")


#Add system equations
model$addSystem(
  dx1 ~ (A * Rainfall - (3 / K1) * x1) * dt + sigma*dw1,

  dx2 ~ (3/K1* x1
        - (3/K2)*x2
        - (3/K_overflow)*(x2/(1+exp(-a1*(x2-b1)))))*dt
        + sigma*dw2,

  dx3 ~ ((3 / K_overflow) * (x2/(1+exp(-a1*(x2-b1))))
        - (3/K_tunnel)*(x3/(1+exp(a2*(x3 - b2))))) * dt
        + sigma*dw3,

  dx4 ~ ((3/K_tunnel)*(x3/(1+exp(a2*(x3 - b2))))
        - cf*Pumpflow*x4)*dt
        + sigma*dw4
)




# Add observation equation
model$addObs(
  Y ~ x4
)



model$setVariance(
  Y ~ sigma_Y^2
)


model$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
```

```
)



# Specify parameter initial values and lower/upper bounds in estimation --------
model$setParameter(
  A            = c(initial = 10, lower = 1e-5, upper = 200),
  logsigma     = log(c(initial = 3, lower = 1, upper = 50)),
  K1           = c(initial = 10, lower = 0.1, upper = 100),
  K2           = c(initial = 10, lower = 0.1, upper = 100),
  K_overflow   = c(initial = 10, lower = 0.1, upper = 100),
  K_tunnel     = c(initial = 10, lower = 0.1, upper = 100),
  a1           = c(initial = 5, lower = 0, upper = 100),
  a2           = c(initial = 5, lower = 0, upper = 100),
  b1           = c(initial = 5, lower = 0, upper = 100),
  b2           = c(initial = 15, lower = 0, upper = 100),
  cf           = c(initial = 1.3, lower = 0, upper = 100),
  logsigma_Y   = log(c(initial = 3, lower = 1, upper = 50))
)



# Add vector input
model$addInput(Rainfall, Pumpflow)



# Set initial state mean and covariance
model$setInitialState(list(mean = c(0,0,0,0), 0.1*diag(4)))


# Carry out estimation with default settings (extended Kalman filter)
fit <- model$estimate(
  data = .data,
  method = "ekf",
  compile = TRUE
)


fit$par.fixed
fit$nll

fit$cov.fixed
cov2cor(fit$cov.fixed)



# Perform full prediction without data update
pred.list = model$predict(data=.data,
                          k.ahead=1e6,
```

```
                        method="ekf",
)

summary(fit)

ggplot() +
  geom_line(data=pred.list$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
```

```
# storage for fitted parameters
df_patameters <- data.frame(
    A = c(0,0,0,0,0,0),
    logsigma = c(0,0,0,0,0,0),
    K1 = c(0,0,0,0,0,0),
    K2 = c(0,0,0,0,0,0),
    K_overflow = c(0,0,0,0,0,0),
    K_tunnel = c(0,0,0,0,0,0),
    a1 = c(0,0,0,0,0,0),
    a2 = c(0,0,0,0,0,0),
    b1 = c(0,0,0,0,0,0),
    b2 = c(0,0,0,0,0,0),
    cf = c(0,0,0,0,0,0),
    logsigma_Y = c(0,0,0,0,0,0)
)


df_patameters[1,] <- as.numeric(fit$par.fixed)

# 2.3.4 - fit to all data ----------------------------------------------------


.data2 <- data.frame(
  t = df.event2$t,
  Rainfall = 60*as.numeric(df.event2$Rainfall/1000),
  Pumpflow = 60*as.numeric(df.event2$Pumpflow)/1000,
  Y = as.numeric(df.event2$Volume)/1000
)
```

```
.data3 <- data.frame(
  t = df.event3$t,
  Rainfall = 60*as.numeric(df.event3$Rainfall/1000),
  Pumpflow = 60*as.numeric(df.event3$Pumpflow)/1000,
  Y = as.numeric(df.event3$Volume)/1000
)

.data4 <- data.frame(
  t = df.event4$t,
  Rainfall = 60*as.numeric(df.event4$Rainfall/1000),
  Pumpflow = 60*as.numeric(df.event4$Pumpflow)/1000,
  Y = as.numeric(df.event4$Volume)/1000
)

.data5 <- data.frame(
  t = df.event5$t,
  Rainfall = 60*as.numeric(df.event5$Rainfall/1000),
  Pumpflow = 60*as.numeric(df.event5$Pumpflow)/1000,
  Y = as.numeric(df.event5$Volume)/1000
)

.data6 <- data.frame(
  t = df.event6$t,
  Rainfall = 60*as.numeric(df.event6$Rainfall/1000),
  Pumpflow = 60*as.numeric(df.event6$Pumpflow)/1000,
  Y = as.numeric(df.event6$Volume)/1000
)
```

```
fit2 <- model$estimate(
  data = .data2,
  method = "ekf",
  compile = TRUE,
  control = list(rel.tol = 1e-3, step.size = 0.1)
)
df_patameters[2,] <- as.numeric(fit2$par.fixed)

fit3 <- model$estimate(
  data = .data3,
  method = "ekf",
  compile = TRUE,
  control = list(rel.tol = 1e-3, step.size = 0.1)
)
df_patameters[3,] <- as.numeric(fit3$par.fixed)

fit4 <- model$estimate(
  data = .data4,
  method = "ekf",
  compile = TRUE,
  control = list(rel.tol = 1e-3, step.size = 0.1)
```

```
)
fit4$par.fixed
df_patameters[4,] <- as.numeric(fit4$par.fixed)

fit5 <- model$estimate(
  data = .data5,
  method = "ekf",
  compile = TRUE,
  control = list(rel.tol = 1e-3, step.size = 0.1)
)
df_patameters[5,] <- as.numeric(fit5$par.fixed)

fit6 <- model$estimate(
  data = .data6,
  method = "ekf",
  compile = TRUE,
  control = list(rel.tol = 1e-3, step.size = 0.1)
)

df_patameters[6,] <- as.numeric(fit6$par.fixed)



# fit average of parameters to all events ------------------------------------
```

```
# Create model object
model1 = ctsmTMB$new()


# Set name of model (and the created .cpp file)
model1$setModelname("larger_real_life_case_event1")


#Add system equations
model1$addSystem(
  dx1 ~ (A * Rainfall - (3 / K1) * x1) * dt + sigma*dw1,

  dx2 ~ (3/K1* x1
         - (3/K2)*x2
         - (3/K_overflow)*(x2/(1+exp(-a1*(x2-b1)))))*dt
  + sigma*dw2,

  dx3 ~ ((3 / K_overflow) * (x2/(1+exp(-a1*(x2-b1))))
         - (3/K_tunnel)*(x3/(1+exp(a2*(x3 - b2))))) * dt
  + sigma*dw3,
```

```
  dx4 ~ ((3/K_tunnel)*(x3/(1+exp(a2*(x3 - b2))))
        - cf*Pumpflow*x4)*dt
  + sigma*dw4
)




# Add observation equation
model1$addObs(
  Y ~ x4
)



model1$setVariance(
  Y ~ sigma_Y^2
)


model1$setAlgebraics(
  sigma ~ exp(logsigma),
  sigma_Y ~ exp(logsigma_Y)
)




# Specify parameter initial values and lower/upper bounds in estimation --------
model1$setParameter(
  A            = mean(df_patameters$A),
  logsigma     = mean(df_patameters$logsigma),
  K1           = mean(df_patameters$K1),
  K2           = mean(df_patameters$K2),
  K_overflow   = mean(df_patameters$K_overflow),
  K_tunnel     = mean(df_patameters$K_tunnel),
  a1           = mean(df_patameters$a1),
  a2           = mean(df_patameters$a2),
  b1           = mean(df_patameters$b1),
  b2           = mean(df_patameters$b2),
  cf           = mean(df_patameters$cf),
  logsigma_Y   = mean(df_patameters$logsigma_Y)
)




# Add vector input
model1$addInput(Rainfall, Pumpflow)
```

```
# Set initial state mean and covariance
model1$setInitialState(list(mean = c(0,0,0,0), 0.1*diag(4)))


pred.list1 = model1$predict(data=.data,
                        k.ahead=1e6,
                        method="ekf",
)

pred.list2 = model1$predict(data=.data2,
                         k.ahead=1e6,
                         method="ekf",
)

pred.list3 = model1$predict(data=.data3,
                         k.ahead=1e6,
                         method="ekf",
)

pred.list4 = model1$predict(data=.data4,
                         k.ahead=1e6,
                         method="ekf",
)

pred.list5 = model1$predict(data=.data5,
                         k.ahead=1e6,
                         method="ekf",
)

pred.list6 = model1$predict(data=.data6,
                         k.ahead=1e6,
                         method="ekf",
)


p.model.mean.1 <- ggplot() +
  geom_line(data=pred.list1$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
p.model.mean.1

p.model.mean.2 <- ggplot() +
  geom_line(data=pred.list2$observations, aes(x=t.j, y=Y, color="Model")) +
```

```
  geom_line(data = .data2, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
p.model.mean.2


p.model.mean.3 <- ggplot() +
  geom_line(data=pred.list3$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data3, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
p.model.mean.3

p.model.mean.4 <- ggplot() +
  geom_line(data=pred.list4$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data4, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
p.model.mean.4

p.model.mean.5 <- ggplot() +
  geom_line(data=pred.list5$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data5, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
```

```
  )
p.model.mean.5

p.model.mean.6 <- ggplot() +
  geom_line(data=pred.list6$observations, aes(x=t.j, y=Y, color="Model")) +
  geom_line(data = .data6, aes(x=t, y=Y, color="Observations")) +
  scale_color_manual(values = c("Model" = "black", "Observations" = "red")) +
  labs(x="Time", y="Volume", color=" ") +
  theme_classic() +
  theme(
    axis.title = element_text(size = 20),
    axis.text = element_text(size = 15),
    legend.title = element_text(size = 20),
    legend.text = element_text(size = 15)
  )
p.model.mean.6
```